PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL
ESCOLA POLITÉCNICA
CURSO DE BACHARELADO EM ENGENHARIA DE SOFTWARE
AGES – AGÊNCIA EXPERIMENTAL DE ENGENHARIA DE SOFTWARE

GUILHERME ANTONIO ARGILAR

MEMORIAL DE ATUAÇÃO NA AGÊNCIA EXPERIMENTAL DE ENGENHARIA DE SOFTWARE - PERÍODO 2023 A 2026

AGES I, II, III e IV

Porto Alegre
2026

AGRADECIMENTOS

Primeiramente, gostaria de agradecer ao meu pai, Adalvio Antonio Argilar, à minha mãe, Custódia Beatriz Argilar, e aos meus irmãos, Rafael Francisco Argilar e Barbara Leticia Argilar, pelo amor incondicional e pela presença constante, sempre me apoiando ao longo da minha jornada, prezando pela minha formação e me dando condições para tal.

Também agradeço à PUCRS pela construção e manutenção do curso de Bacharelado em Engenharia de Software.

Por fim, agradeço a todos os professores e professoras do Bacharelado em Engenharia de Software que tive o privilégio de conhecer no decorrer do curso. Agradeço também à equipe da AGES por proporcionar uma experiência única vivenciada pelos alunos do curso.

RESUMO

No presente documento é apresentada a experiência do aluno Guilherme Argilar durante sua passagem pela Agência Experimental de Engenharia de Software (AGES) da PUCRS, ambiente essencial para o intercâmbio de conhecimento e o aprendizado prático no curso de Bacharelado em Engenharia de Software. A AGES é composta por quatro disciplinas oferecidas ao longo da graduação, nas quais os alunos assumem responsabilidades progressivamente maiores conforme sua senioridade: como AGES I, executam tarefas de programação; como AGES II, atuam na análise de dados da aplicação; como AGES III, no desenvolvimento da arquitetura de software; e, como AGES IV, na gestão da equipe do projeto. Ao longo de sua graduação, o autor participou dos quatro módulos da Agência, vivenciando diferentes papéis no processo de desenvolvimento de software e na dinâmica do trabalho em equipe. Este documento narra sua jornada ao longo do curso, com foco nas experiências dos semestres 2023/1, 2024/1, 2025/1 e 2026/1, durante os quais esteve envolvido nos projetos Focus, Decola, Totem Interativo e Estudify, respectivamente. Em cada módulo, são destacados os aprendizados, os desafios enfrentados e a evolução das responsabilidades dentro da AGES.

PALAVRAS CHAVES: AGES, Engenharia de Software, Aprendizado, Processo de Desenvolvimento de Software, Projeto.

LISTA DE ILUSTRAÇÕES

Figura 1 – Time do projeto Focus 13
Figura 2 – Modelo entidade-relacionamento 14
Figura 3 – Diagrama do padrão arquitetural MVC 15
Figura 4 – Diagrama geral do sistema 16
Figura 5 – Protótipos das telas principais: inicial, frases, objetivos e notificação 17
Figura 6 – Protótipo das telas de cadastro 17
Figura 7 – Time do projeto Decola 27
Figura 8 – Modelo relacional do banco de dados 29
Figura 9 – Modelo de autenticação 30
Figura 10 – Tela de página inicial 31
Figura 11 – Tela de administração 32
Figura 12 – Tela de áreas 32
Figura 13 – Tela de atividade de múltipla escolha 33
Figura 14 – Tela de atividade de conteúdo externo 33
Figura 15 – Time do projeto Totem Interativo 41
Figura 16 – Modelo relacional do banco de dados 43
Figura 17 – Arquitetura do Totem Interativo e fluxo de comunicação do *frontend* 44
Figura 18 – Tela inicial com mapa interativo 47
Figura 19 – Menus de categorias e expositores 47
Figura 20 – Busca de expositores 47
Figura 21 – Tela de configurações do administrador 48
Figura 22 – Mapa desenvolvido 53
Figura 23 – Primeira versão do mapa do Tecnopuc 54
Figura 24 – Versão final do mapa do Tecnopuc 56
Figura 25 – Time do projeto Estudify 61
Figura 26 – Modelo conceitual Estudify 62
Figura 27 – Modelo lógico relacional Estudify 63
Figura 28 – Diagrama de fluxo da aplicação 64
Figura 29 – Diagrama de *deploy* da aplicação 65
Figura 30 – Protótipos das telas do aplicativo *mobile* 66

Figura 31 – Painel administrativo: gestão de questões 67
Figura 32 – Painel administrativo: gestão de simulados 67
Figura 33 – Painel administrativo: revisão de questões importadas via CSV . . . 68

LISTA DE SIGLAS

ACID	Atomicity, Consistency, Isolation, Durability
AGES	Agência Experimental de Engenharia de Software
API	Application Programming Interface
APK	Android Package Kit
AWS	Amazon Web Services
CDN	Content Delivery Network
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DDD	Domain Driven Development
DTO	Data Transfer Object
EC2	Elastic Compute Cloud
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IHC	Interação Humano-Computador
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
ORM	Object-Relational Mapping
PR	Pull Request
PUCRS	Pontifícia Universidade Católica do Rio Grande do Sul
REST	Representational State Transfer
SoC	Separation of Concerns
SQL	Structured Query Language
SSL	Secure Sockets Layer
UI	User Interface

URL	Uniform Resource Locator
US	User Stories
UUID	Universally Unique Identifier
UX	User Experience

SUMÁRIO

1 APRESENTAÇÃO DA TRAJETÓRIA DO ALUNO 10

2 AGES I — PROJETO FOCUS 12
2.1 Introdução 12
2.2 Desenvolvimento do Projeto 13
2.2.1 Repositório do Código Fonte do Projeto 13
2.2.2 Banco de Dados Utilizado 13
2.2.3 Arquitetura Utilizada 14
2.2.4 Protótipos das Telas Desenvolvidas 16
2.2.5 Tecnologias Utilizadas 17
2.3 Atividades Desempenhadas Pelo Aluno no Projeto 18
2.3.1 Sprint 0 18
2.3.2 Sprint 1 19
2.3.3 Sprint 2 20
2.3.4 Sprint 3 21
2.3.5 Sprint 4 23
2.4 Conclusão 24

3 AGES II — DECOLA 26
3.1 Introdução 26
3.2 Desenvolvimento do Projeto 27
3.2.1 Repositório do Código Fonte do Projeto 27
3.2.2 Banco de Dados Utilizado 27
3.2.3 Arquitetura Utilizada 29
3.2.3.1 Backend 29
3.2.3.2 Frontend 31
3.2.4 Protótipos das Telas Desenvolvidas 31
3.2.5 Tecnologias Utilizadas 33
3.3 Atividades Desempenhadas Pelo Aluno no Projeto 34
3.3.1 Sprint 0 34
3.3.2 Sprint 1 35

3.3.3 Sprint 2 . . . 36
3.3.4 Sprint 3 . . . 37
3.3.5 Sprint 4 . . . 37
3.4 Conclusão . . . 38

4 AGES III — TOTEM INTERATIVO . . . 40
4.1 Introdução . . . 40
4.2 Desenvolvimento do Projeto . . . 41
4.2.1 Repositório do Código Fonte do Projeto . . . 41
4.2.2 Banco de Dados Utilizado . . . 42
4.2.3 Arquitetura Utilizada . . . 43
4.2.3.1 Frontend . . . 44
4.2.3.2 Backend . . . 45
4.2.4 Protótipos das Telas Desenvolvidas . . . 46
4.2.5 Tecnologias Utilizadas . . . 48
4.3 Atividades Desempenhadas Pelo Aluno no Projeto . . . 49
4.3.1 Sprint 0 . . . 49
4.3.2 Sprint 1 . . . 50
4.3.3 Sprint 2 . . . 51
4.3.4 Sprint 3 . . . 52
4.3.5 Sprint 4 . . . 55
4.4 Conclusão . . . 58

5 AGES IV — ESTUDIFY . . . 60
5.1 Introdução . . . 60
5.2 Desenvolvimento do Projeto . . . 61
5.2.1 Repositório do Código Fonte do Projeto . . . 61
5.2.2 Banco de Dados Utilizado . . . 62
5.2.3 Arquitetura Utilizada . . . 63
5.2.4 Protótipos das Telas Desenvolvidas . . . 65
5.2.5 Tecnologias Utilizadas . . . 68
5.3 Atividades Desempenhadas Pelo Aluno no Projeto . . . 69
5.3.1 Sprint 0 . . . 69

5.3.2 Sprint 1 . . . . . **72**
5.3.3 Sprint 2 . . . . . **74**
5.3.4 Sprint 3 . . . . . **76**

Referências **79**

1 APRESENTAÇÃO DA TRAJETÓRIA DO ALUNO



Em 2017, iniciei minha jornada na Pontifícia Universidade Católica do Rio Grande do Sul ~~(Pontifícia Universidade Católica do Rio Grande do Sul~~ (PUCRS)~~)~~, no curso de Engenharia de Computação. Foi nesse momento que tive meu primeiro contato com programação e me encantei com as possibilidades que ela proporcionava. Em 2018, durante um evento de Portas Abertas da PUCRS, conheci o curso de Engenharia de Software e percebi que ele abrangia muito mais a parte de programação do que a Engenharia de Computação. Decidi solicitar a troca de curso em 2019.

No segundo semestre de 2019, tentei cursar a disciplina de Agência Experimental de Engenharia de Software (AGES) I pela primeira vez. No entanto, logo no primeiro mês, acabei trancando a disciplina, pois senti que não estava conseguindo contribuir e enfrentei algumas questões pessoais. Tive minha segunda oportunidade como AGES I em 2023/01 e senti-me muito mais preparado para enfrentá-la do que antes. Nesse momento, já possuía conhecimentos mais aprofundados e pude auxiliar ativamente a equipe.

Em 2024/01, durante minha segunda experiência com a AGES, pude contribuir ainda mais para o projeto, sem a necessidade de suporte constante de colegas de semestres superiores. Meu crescimento me permitiu assumir responsabilidades em tarefas mais complexas, contribuindo para acelerar o processo de desenvolvimento e marcando minha evolução como estudante de Engenharia de Software.

Em 2025/01, assumi o papel de AGES III. Nesta posição, pude aplicar todo o conhecimento acumulado nos semestres anteriores, agora como mentor e líder técnico. A responsabilidade de orientar AGES I e II, tomar decisões arquiteturais e garantir a qualidade das entregas consolidou minha transformação de aprendiz para orientador. A capacidade de resolver problemas críticos sob pressão demonstraram que estava pronto para os desafios do mercado de trabalho.

Em 2026/01, assumi o papel de AGES IV no projeto Estudify. Foi a primeira vez que olhei para um projeto inteiramente pela ótica da gestão, e não mais pela da execução. Em vez de pegar tarefas para desenvolver, meu trabalho passou a ser escrever e detalhar as histórias, distribuí-las entre os integrantes da minha *squad*, acompanhar o andamento das entregas e remover impedimentos quando apareciam. Aprender a

confiar no time e a delegar as tarefas, ao invés de tentar executá-las eu mesmo, foi um dos maiores desafios dessa experiência. Esse semestre fechou o ciclo da AGES para mim, consolidando uma trajetória que começou com dúvidas sobre onde eu poderia contribuir e terminou com a responsabilidade de garantir que um time inteiro chegasse junto nas entregas.

2 AGES I — PROJETO FOCUS

Esta seção busca apresentar a passagem do aluno como AGES I pelo Projeto Focus. Aqui estão descritos os artefatos entregues, a atuação ao longo das sprints e os pontos de melhoria identificados.

2.1 Introdução

O Projeto foi concebido através de uma “dor” que o *stakeholder*, Lucas Savaris, e pessoas próximas a ele tinham: o mau aproveitamento de seu tempo por conta das viciantes redes sociais.

O projeto tem como objetivo auxiliar o usuário a reduzir o tempo de uso do celular e tornar o seu uso mais produtivo. Com esse objetivo, foi proposto um aplicativo móvel que permite ao usuário definir o tempo que deseja passar utilizando o celular, e receber notificações personalizadas quando atingir o limite estabelecido.

Além disso, o aplicativo também apresentará uma barra de progresso indicando o tempo restante com base no tempo de uso definido pelo usuário. Caso o usuário ultrapasse o limite estabelecido, o aplicativo ativará uma terceira funcionalidade denominada *Greyscale*, na qual a tela do celular ficará em escala de cinza.

A execução do projeto ocorreu no primeiro semestre de 2023, entre os dias 27 de fevereiro e 21 de junho, pelos estudantes do bacharelado de Engenharia de Software em diversas etapas da AGES, com orientação do professor Marcelo Hideki Yamaguti. A foto do time responsável pelo projeto pode ser vista na Figura 1.

Figura 1 – Time do projeto Focus



Fonte: Wiki do projeto

2.2 Desenvolvimento do Projeto

Esta seção apresenta informações relativas ao desenvolvimento do projeto: localização do código-fonte e documentação, arquitetura e tecnologias utilizadas, e protótipos de tela desenvolvidos. Toda a documentação do projeto pode ser encontrada na wiki do mesmo: `https://tools.ages.pucrs.br/projeto-focus/wiki`.

2.2.1 Repositório do Código Fonte do Projeto

Todo o código do projeto encontra-se distribuído em dois repositórios:

- código do *backend*: `https://tools.ages.pucrs.br/projeto-focus/backend`;
- código do *frontend*: `https://tools.ages.pucrs.br/projeto-focus/frontend`.

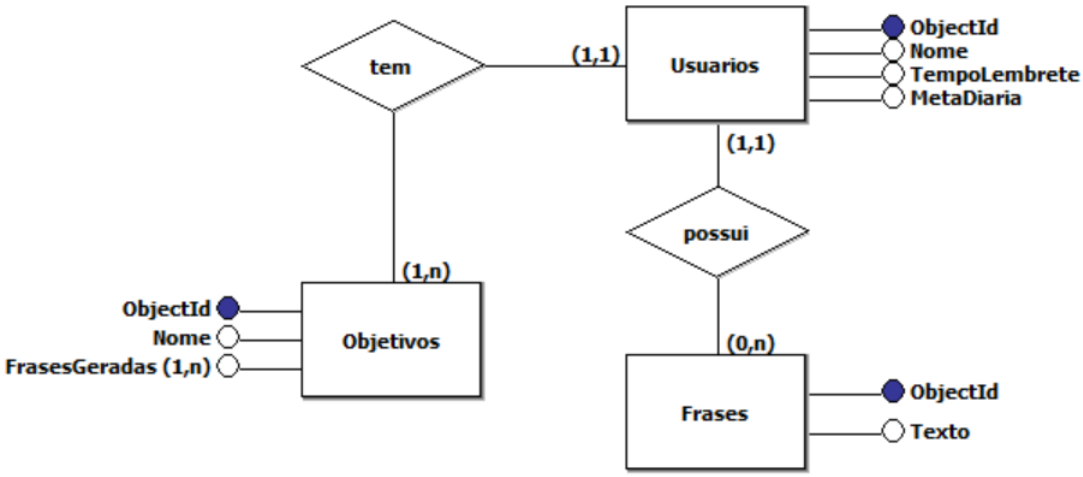
Há, ainda, um terceiro repositório, usado somente para abrigar a wiki do projeto. A wiki encontra-se em: `https://tools.ages.pucrs.br/projeto-focus/wiki`.

2.2.2 Banco de Dados Utilizado

Para a realização do projeto, escolhemos utilizar o banco de dados não relacional Firebase. A razão pela escolha deste modelo de banco de dados se deu principalmente pela escolha do time em aprender novas tecnologias, pela facilidade de inte-

gração com o *backend* da aplicação e também devido ao projeto possuir somente uma entidade a ser armazenada, que é o usuário. A Figura 2 apresenta o modelo de dados da aplicação.

Figura 2 – Modelo entidade-relacionamento



Fonte: Wiki do projeto

Durante a apresentação do projeto, o *stakeholder* afirmou que não haveria a necessidade de armazenar os dados, pois caso fosse necessário o usuário poderia apenas reescrevê-los. Após uma conversa com o time, conseguimos mostrar como um banco de dados, mesmo que o mais simplificado possível, poderia agregar no seu aplicativo.

2.2.3 Arquitetura Utilizada

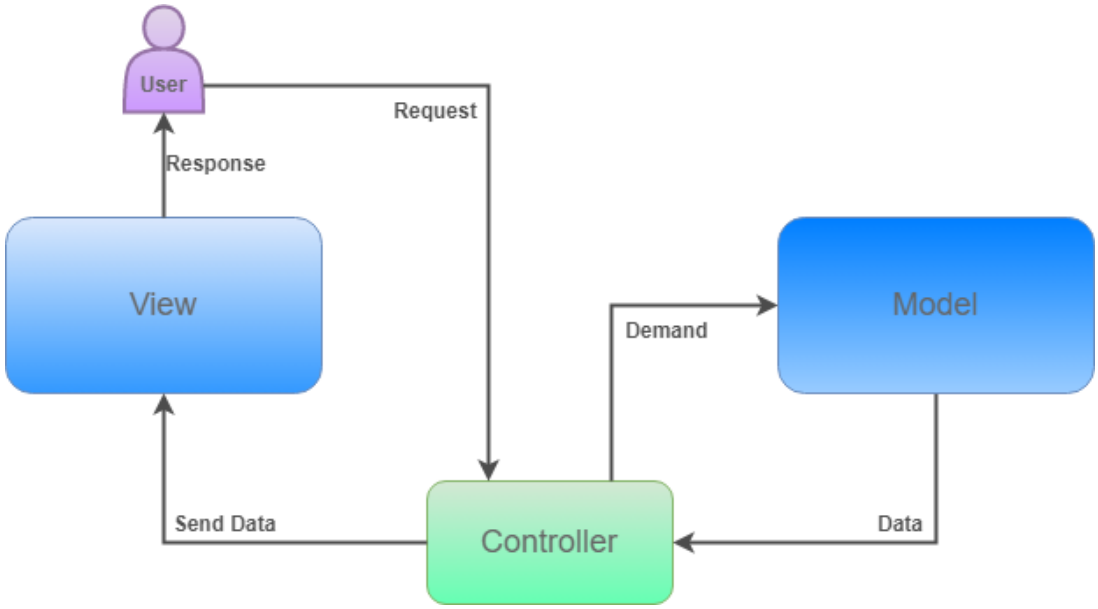
Este projeto foi desenvolvido utilizando a arquitetura Model-View-Controller (MVC) no *front-end*. Esse padrão arquitetural tem como objetivo separar as responsabilidades da aplicação em três camadas distintas, o que facilita a manutenção, a organização e o reuso do código. As camadas são:

- ***Model*:** camada de manipulação de dados, responsável pela leitura, escrita e validação das informações. É completamente independente das demais camadas;
- ***View*:** camada de interação com o usuário, que representa a parte visual da aplicação e é alimentada pela camada *Controller* somente quando necessário;

- ***Controller*:** camada responsável por receber todas as requisições do usuário. Seus métodos controlam qual *Model* utilizar e qual *View* será apresentada ao usuário.

A Figura 3 ilustra a organização dessas camadas no padrão MVC.

Figura 3 – Diagrama do padrão arquitetural MVC

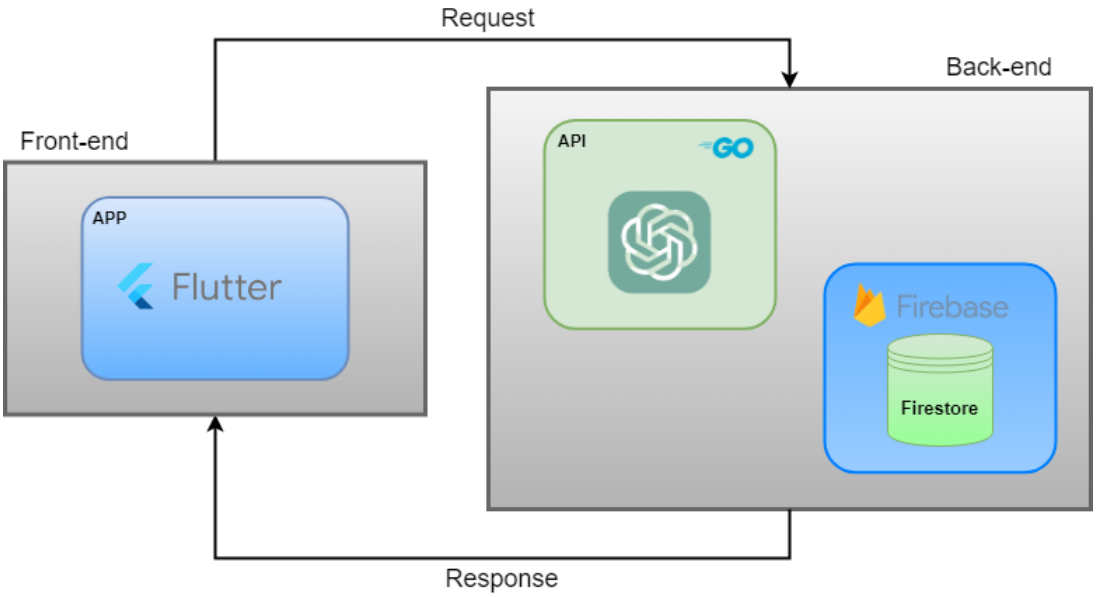


Fonte: Wiki do projeto

No *front-end*, desenvolvido em Flutter, optou-se pela adoção do padrão MVC, enquanto no *back-end* foram utilizadas as chamadas diretas ao Firebase. A comunicação da aplicação ainda contempla uma Application Programming Interface (API) externa do ChatGPT, implementada em Go, responsável por integrar recursos de chat baseados em inteligência artificial. Essa API é utilizada para gerar frases de acordo com os objetivos selecionados pelo usuário.

A Figura 4 apresenta a visão geral da arquitetura, evidenciando como ocorre a comunicação entre as partes do sistema. O aplicativo desenvolvido em Flutter atua como ponto central de interação com o usuário e se comunica com dois serviços distintos: o Firebase, utilizado para autenticação e armazenamento dos dados; e a API externa do ChatGPT, responsável por fornecer as frases motivacionais geradas por inteligência artificial. Dessa forma, o *front-end* concentra a lógica de apresentação e orquestra as requisições, enquanto os serviços de *back-end* ficam responsáveis, respectivamente, pela persistência das informações e pelo processamento das respostas baseadas em IA.

Figura 4 – Diagrama geral do sistema



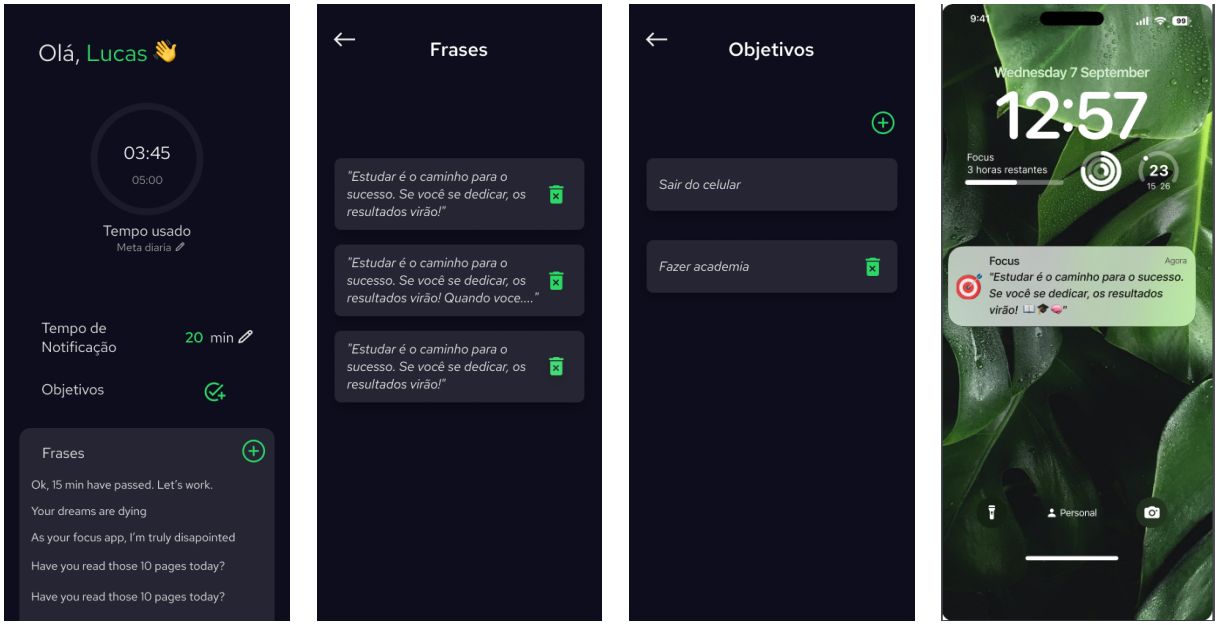
Fonte: Wiki do projeto

Para a infraestrutura, foi utilizada a plataforma de *cloud computing* Elastic Compute Cloud (EC2), da Amazon, com o propósito de alocar os *runners* do GitLab e manter a *pipeline* do projeto em execução. Informações adicionais, bem como os diagramas de arquitetura e de *deploy*, podem ser encontradas na página de arquitetura do projeto: `https://tools.ages.pucrs.br/projeto-focus/wiki/-/wikis/arquitetura`.

2.2.4 Protótipos das Telas Desenvolvidas

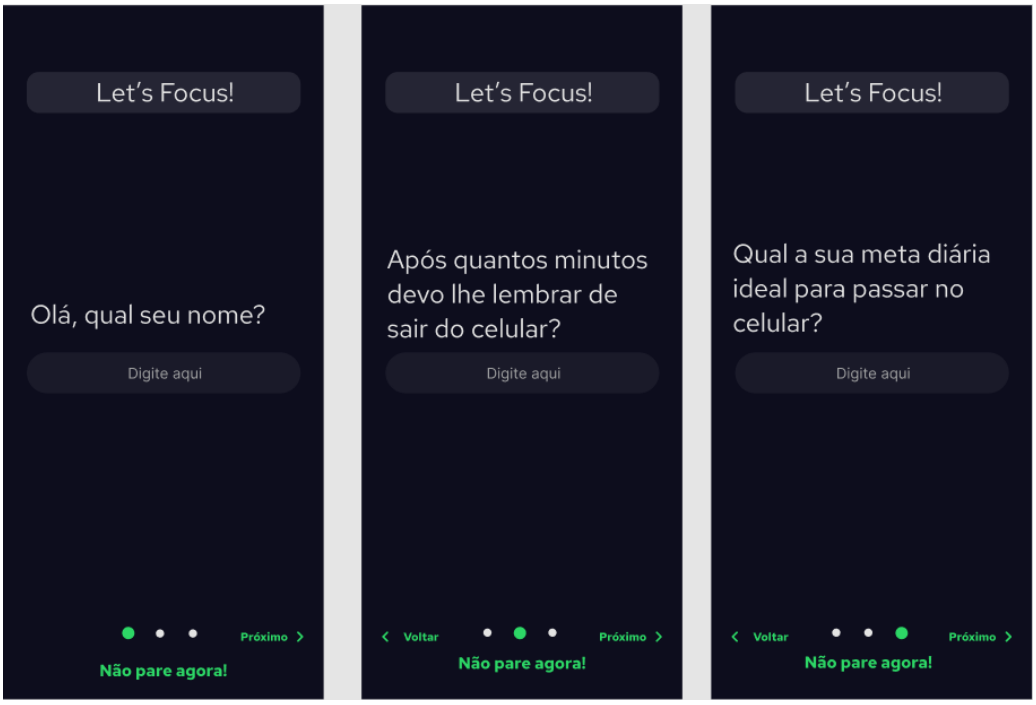
Os protótipos deste projeto estão disponíveis no endereço a seguir: `https://tools.ages.pucrs.br/projeto-focus/wiki/-/wikis/design`. As Figuras 5 e 6 apresentam uma amostra das telas desenvolvidas. A Figura 5 contempla o fluxo principal de uso, com a tela inicial, a exibição das frases motivacionais, a seleção de objetivos e a configuração de notificações. Já a Figura 6 abrange o fluxo de cadastro e autenticação do usuário.

Figura 5 – Protótipos das telas principais: inicial, frases, objetivos e notificação



Fonte: Wiki do projeto

Figura 6 – Protótipo das telas de cadastro



Fonte: Wiki do projeto

2.2.5 Tecnologias Utilizadas

Nesta seção será abordado um pouco sobre as tecnologias utilizadas ao longo do projeto.

- **Flutter** (Google, 2017): O Flutter foi criado pela Google, sendo um *framework* mais voltado para desenvolvimento de aplicativos móveis. Ele permite a criação de aplicativos compilados nativamente para iOS, Android, Windows, Mac, Linux e Web. Utilizando a linguagem de programação Dart, o Flutter se diferencia de outros *frameworks* por utilizar uma abordagem de desenvolvimento baseada em *Widgets*, que são elementos gráficos reutilizáveis e personalizáveis.

- **Firebase** (Google, 2011): O Firebase é uma plataforma de desenvolvimento de aplicativos móveis e web da Google, que fornece diversos recursos para ajudar os desenvolvedores a criar aplicações de forma efetiva, rápida e simples. Ele inclui diversas funcionalidades, incluindo armazenamento de dados em tempo real, autenticação de usuários, hospedagem, mensagens e notificações, análise de dados, entre outros.

- **Go** (Griesemer; Thompson; Pike, 2009): O Go, também conhecido como Golang, é uma linguagem de programação de código aberto criada pelo Google. Ela foi projetada para oferecer eficiência, simplicidade e escalabilidade, tornando-se uma escolha popular para o desenvolvimento de aplicativos de servidor e sistemas de rede.

2.3 Atividades Desempenhadas Pelo Aluno no Projeto

2.3.1 Sprint 0

O projeto teve início com uma reunião com o cliente, onde foi apresentada a ideia do projeto, como surgiu e as inspirações que foram consideradas para realizá-lo, além de terem sido esclarecidas as dúvidas que foram surgindo. Juntamente com o Lucas, os AGES IV do projeto, Rafael Cardoso e Vitor Hugo, montaram *sketches* de cada tela, alterando conforme as ideias do time iam surgindo e sendo validadas pelo próprio *stakeholder*.

Após a reunião de apresentação do projeto, o time se juntou para definir as tecnologias que seriam utilizadas no desenvolvimento. Em seguida, dividimo-nos em *squads* de quatro integrantes, com o objetivo de gerar diferentes ideias para as telas do aplicativo e ter mais opções. Cada integrante, especialmente os AGES I, criou telas de acordo com sua compreensão dos requisitos do *stakeholder*.

Após a criação das primeiras ideias para o aplicativo, o time se reuniu novamente para selecionar os principais pontos positivos de cada *mockup*, a fim de evitar desperdício de trabalho.

Com uma base sólida do que deveria ser feito, os gerentes do projeto decidiram dividir as telas entre as *squads*. Meu *squad* foi responsável pelos objetivos e também fui encarregado de iniciar o *style guide* do projeto. O *squad* organizou um horário para discutir suas ideias, a fim de chegar a uma tela final. Como não sabia quais componentes os outros *squads* poderiam precisar, criei um *style guide* mais abrangente, incluindo alguns componentes específicos utilizados nas telas de objetivos.

Nas últimas aulas da sprint, como o *squad* havia concluído as telas, cada um ajudou os demais conforme possível. Como eu já tinha conhecimento em animações e *flows* no Figma, trabalhei em conjunto com o *Squad* do Fogo, auxiliando principalmente na criação de componentes que mudam de cor ao serem clicados, bem como no botão *switch*, utilizados nas telas de objetivos e na tela principal.

2.3.2 Sprint 1

Ao final da Sprint 0, foi realizada uma reunião com o *stakeholder*, na qual ele aprovou a grande maioria das telas. Contudo, ele solicitou algumas alterações no fluxo e na tela inicial do aplicativo. Mais especificamente, foi solicitada a adição de dois novos botões na tela inicial: um que levaria à lista de frases e outro que permitiria a adição rápida de uma nova frase.

Na aula seguinte da apresentação, os *squads* foram divididos novamente para desenvolver as seguintes tarefas: o cadastro do usuário, a tela inicial e a refatoração das telas no Figma. Meu *squad* ficou responsável pela tela inicial. Junto com o AGES III Leonardo José Machado Canto, dividimos a tarefa da tela inicial em duas partes iniciais: os botões e o gráfico da meta diária. Cada dupla ficou responsável por uma dessas partes, e eu escolhi ser responsável pela criação dos demais componentes.

Como não tive tempo de estudar Flutter durante a Sprint 0, precisei focar exclusivamente nos estudos dirigidos durante a primeira semana da Sprint 1. Na primeira semana, junto com o Leonardo, criamos um esboço inicial, já que ainda estávamos aguardando o design do outro *squad* ser finalizado e aprovado pelo cliente. Infelizmente, nosso *squad* teve problemas de comunicação e falta de participação de alguns

integrantes em suas tarefas, o que sobrecarregou nosso *Squad Leader*, que acabou ajudando em várias partes da criação das telas.

Porém, a tela inicial só foi aprovada pelo *stakeholder* quatro dias antes do *Freezing*. Logo no mesmo dia, comuniquei ao time que tentaria criar algo em Flutter parecido com o que o cliente havia aprovado, para que pudéssemos discutir possíveis alterações e melhorias na User Interface (UI) do aplicativo. Acabei desenvolvendo parte do trabalho que, a princípio, não seria minha responsabilidade, mas se mostrou necessário. Neste dia, foram criados quase todos os componentes que compõem a tela inicial.

Durante a última aula antes da entrega, nos reunimos com os líderes do *squad* responsável pela refatoração das telas e definimos alguns pontos importantes. O principal é que o *mockup* aprovado havia sido feito pelo próprio *stakeholder*, que demonstrou ser proativo, mas foi criado em um tamanho muito superior ao das outras telas, o que tornou inviável transpor fidedignamente essa tela para a aplicação.

Dois dias antes do *Freezing*, ainda tínhamos pendências referentes à padronização de alguns elementos. Como havia sido solicitado pelo Leonardo, mas ninguém havia se mostrado interessado em pegar as tarefas, senti-me no dever de entregar os dois últimos itens que restavam para finalizar.

A demora na definição da tela acabou atrasando bastante a entrega do meu *squad*, e junto com outros fatores, como a falta de um ambiente de testes e um banco de dados para a retirada das informações do cadastro, nossa tarefa acabou por não ser aceita. Apesar disso, fico contente com o trabalho que realizei. Como mencionado anteriormente, eu não tinha conhecimento algum em Flutter e mesmo assim consegui estudar e contribuir muito durante a Sprint.

2.3.3 Sprint 2

Após a primeira entrega de código ao cliente, percebemos que era necessário fazer algumas mudanças significativas. Não conseguimos entregar nenhum dos entregáveis da sprint anterior e sabíamos que precisávamos mudar nossa abordagem e nos concentrar mais para que o projeto avançasse. Na primeira semana da nova sprint, os dois AGES IV do projeto decidiram alterar a forma como trabalharíamos. Eles dividiram as User Stories (US) em pequenas tarefas e atribuíram uma tarefa a

cada membro da equipe, para facilitar o acompanhamento e a assistência.

Eu me ofereci para assumir a tarefa de coletar o tempo de tela do usuário. Sabia que seria desafiador, mas isso também me motivou, pois se conseguisse desenvolvê-la, muitas funcionalidades do aplicativo estariam bem encaminhadas. A solução que encontrei foi utilizar a biblioteca `screen_state` do Flutter, que monitora a tela do celular para verificar se está bloqueada ou desbloqueada. Além disso, utilizei o plugin `Pausable_Timer` para controlar o tempo que o usuário passou com a tela desbloqueada, pausando o *timer* quando a tela era bloqueada.

No entanto, ainda precisava enviar o tempo coletado para o banco de dados. Como o banco de dados estava sendo desenvolvido apenas nessa mesma sprint, tive pouco tempo para aprender mais sobre o Flutter, pesquisar sobre minha tarefa e entender os conceitos básicos de como nosso Firebase estava funcionando.

Durante o último final de semana da sprint, toda a equipe se concentrou em concluir suas tarefas e entregar o máximo possível para gerar um Android Package Kit (APK) a ser entregue ao cliente. Pedi a um dos AGES II do projeto que me ajudasse a coletar o tempo e enviá-lo para o banco de dados, e também para me mostrar como obter os dados referentes ao tempo de notificação configurado pelo usuário. Depois de algumas horas tentando, percebemos que não seria possível concluir essa parte, pois isso afetaria muito as tarefas dos outros membros da equipe, exigindo retrabalho, e não havia tempo suficiente para isso.

No final, conseguimos gerar o APK e entregar tudo o que estava pendente da Sprint 1 e o que foi proposto na Sprint 2, exceto minha tarefa. Isso me deixou um pouco frustrado, mas sabia que, por ser algo mais complexo, os AGES III e IV entenderam e me ajudariam na próxima sprint para que pudesse concluir essa tarefa.

2.3.4 Sprint 3

Durante a primeira semana da Sprint, realizamos uma sessão de *Planning Poker*, que é uma estratégia para estimar o esforço necessário no desenvolvimento das funcionalidades do aplicativo. Desde o início da Sprint 3, eu já sabia qual seria minha tarefa: finalizar a coleta de tempo de tela. Após a sessão de planejamento, comecei a pesquisar artigos sobre assincronicidade para me auxiliar nessa tarefa. Na Sprint 2, identifiquei um problema em que o usuário não tinha um tempo de notificação defi-

nido ao criar sua conta, e devido aos requisitos do `Pausable_Timer`, não era possível passar um valor nulo. Decidi pedir ajuda ao AGES IV Vitor Hugo Garcez Barcellos dos Santos, pois ele havia estudado bastante sobre o assunto em sua última passagem pela AGES.

Marcamos um horário para que eu pudesse mostrar a ele o código e os erros que estavam ocorrendo. Logo de início, percebemos que, com alguns métodos implementados no final da Sprint 2, o problema de assincronicidade se tornou trivial, e seria necessário apenas refatorar partes do código. Além disso, constatamos que, com essas alterações, não seria necessário salvar o tempo dos *timers* no banco, pois o código poderia ser executado em segundo plano e, quando o tempo selecionado pelo usuário expirasse, poderíamos chamar uma função.

Após essa conversa com Vitor, verificamos que o *timer* da tela inicial não era retomado após o usuário desbloquear a tela. Ao analisar a documentação do plugin, constatamos que várias pessoas também enfrentavam esse mesmo problema. Decidimos deixar essa funcionalidade como uma pendência para a última entrega, pois a tarefa de coleta de tempo de tela já estava concluída. Vitor me pediu para fazer a integração do código que envia a notificação, busca uma frase e cria um mecanismo de agendamento da notificação.

Ao analisar essas partes do código, deparei-me com as seguintes situações: o código de notificações não havia sido testado em nenhum ambiente Android e o mecanismo de agendamento de notificação ainda não havia sido implementado. Iniciei o desenvolvimento do mecanismo de agendamento, pois era o problema menos complexo e pude resolvê-lo rapidamente. Em seguida, solicitei ajuda ao AGES I Luis Felipe Copetti Rossi, que havia criado o código para chamar a notificação, para compreender melhor o código e resolver as pendências relacionadas ao Android. Após analisar a documentação da biblioteca, identificamos que seria necessário fazer algumas alterações no Gradle. Assim que adicionamos as dependências solicitadas na documentação, a integração foi concluída.

Durante a fase de *Freezing*, acabei desenvolvendo um *hotfix* para uma requisição no aplicativo que envia notificações. Também acompanhei o restante dos *Pull Requests* e como estavam resolvendo os conflitos e pude aprender bastante.

2.3.5 Sprint 4

Após a entrega da Sprint 3, foram identificados vários *bugs* relacionados às notificações, incluindo:

- Ao realizar qualquer alteração no aplicativo, o *timer* era resetado e uma notificação era enviada.
- Os *timers* não estavam sendo editados corretamente.
- O *timer* da tela principal não funcionava em segundo plano.

Devido ao meu amplo conhecimento sobre o código, fui encarregado de encontrar e resolver esses *bugs*.

Para corrigir o problema de edição dos *timers*, criei uma função no arquivo responsável pela coleta do tempo de tela. Essa função cancelava o *timer* atual e criava um novo *timer*, com a duração correspondente ao novo valor, subtraindo o tempo que já havia passado no *timer* antigo. Em seguida, chamei essa função no local onde a função de edição do tempo estava sendo invocada.

No caso do *bug* relacionado às notificações, percebi que não havia um erro no código em si, mas sim no comportamento do aplicativo. Toda vez que uma ação era executada no app, a tela inicial era recriada, o que também recriava o método de coleta do tempo de tela. Para solucionar esse problema, adicionei um booleano para verificar se os *timers* já haviam sido criados.

Por fim, para resolver o problema do *timer* da tela inicial, foi necessário recriá-lo completamente, utilizando uma biblioteca para contar o tempo e outra para criar uma barra de progresso em formato circular. Devido à complexidade dessa tarefa, consegui concluí-la apenas no penúltimo dia da fase de *Freezing*.

Durante a fase de *Freezing*, novamente me reuni com a equipe para ajudar e também aprender a lidar com conflitos de código. Como eu era um dos poucos capazes de utilizar o emulador do Android Studio para executar o projeto, fiquei responsável por testar e validar se os Pull Request (PR) aceitos não estavam quebrando o projeto. Devido ao grande número de tarefas a serem entregues, enfrentamos vários conflitos, o que atrasou consideravelmente a conclusão do projeto.

No final da fase de *Freezing*, juntei-me aos AGES III do projeto para realizar um *hotfix* no *timer* da tela inicial. Nessa Sprint, eles haviam implementado o uso de *providers*

no código para atualizar as telas dinamicamente, sem a necessidade de reconstruí-las. Utilizamos a mesma técnica para permitir que, quando o usuário desejasse alterar a meta diária, a barra de progresso e o tempo total fossem atualizados dinamicamente, trazendo maior qualidade ao aplicativo.

No dia da entrega para o *stakeholder*, recebi uma mensagem do Vitor pedindo para verificar um *bug* que estava ocorrendo. O *bug* estava relacionado à multiplicação incorreta do tempo de notificação ao ser editado. Devido ao meu amplo envolvimento com essa parte do código ao longo do semestre, estava familiarizado com o local onde poderia encontrar esse erro e corrigi-lo. Tive a oportunidade de enviar o último PR para a *branch Development* antes da apresentação para o *stakeholder*.

Durante a Sprint 4, nossa equipe conseguiu entregar o aplicativo com todas as funcionalidades previstas, superando os desafios e corrigindo os *bugs* identificados. Foi gratificante ver o cliente satisfeito com o resultado final, demonstrando que atendemos às suas expectativas. Além disso, nos orgulhamos de ter entregado um código de qualidade, com as devidas correções e melhorias, garantindo um produto robusto e confiável.

2.4 Conclusão

Minha experiência como AGES I foi muito importante no meu crescimento pessoal e profissional. Foi uma oportunidade de vivenciar de perto a dinâmica de trabalho em equipe multidisciplinar, proporcionando uma visão mais realista do que encontrarei no mercado de trabalho. Acompanhei de perto o processo de desenvolvimento de um software a partir de um código existente, o que foi muito enriquecedor.

Fiquei impressionado com o engajamento e compromisso da equipe com o projeto, especialmente em relação à entrega ao *stakeholder*. Essa dedicação nos permitiu estabelecer uma maior coesão entre os membros da equipe. Além disso, tive a oportunidade de conhecer e interagir com colegas de curso que não conhecia antes, criando laços acadêmicos que levo comigo para os próximos semestres.

A experiência de desenvolver um projeto com entregas iterativas e incrementais, com o compromisso de entregar um sistema funcional ao final, foi muito interessante. Embora na Agência não sejamos “obrigados” a entregar um sistema completo aos clientes no final do semestre, o fato de construir algo que será utilizado por outras

pessoas foi uma motivação poderosa para dar o meu melhor.

Enfrentamos alguns problemas de comunicação dentro da equipe, o que resultou em atrasos na entrega de algumas funcionalidades. O principal obstáculo foi a definição tardia da interface do *backend*. Um aspecto negativo que observei foi a falta de testes de unidade e integração, algo que considero essencial no desenvolvimento de software.

No que se refere à relação entre as disciplinas cursadas e a AGES, acredito que, por ter iniciado a minha caminhada na Agência mais tarde do que esperava, consegui contribuir de forma mais efetiva para o projeto, mesmo atuando como AGES I. As disciplinas que mais me ajudaram nesse sentido foram Fundamentos de Interação Humano-Computador (IHC) e Gerenciamento de Configuração de Software. A primeira me deu uma base importante para compreender as decisões de interface e usabilidade do aplicativo, auxiliando no desenvolvimento das telas do *frontend*. Já a segunda foi fundamental para que eu lidasse com o controle de versão e o fluxo de trabalho com Git, facilitando a integração do meu código ao repositório e a colaboração com o restante da equipe.

No geral, acredito que este semestre me proporcionou um crescimento pessoal significativo. Melhorei minhas habilidades sociais ao interagir com meus colegas de equipe e participar de reuniões com o *stakeholder*. Além disso, expandi meu conhecimento técnico ao desenvolver habilidades em *backend* e *frontend*, o que me abre mais oportunidades no mercado de trabalho.

3 AGES II — DESENVOLVIMENTO DE PLATAFORMA DE ONBOARDING PARA NOVOS COLABORADORES

Esta seção busca apresentar a passagem do aluno como AGES II pelo Projeto Decola. Aqui estão descritos os artefatos entregues, a atuação ao longo das sprints e os pontos de melhoria identificados.

3.1 Introdução

Em um ambiente corporativo que evolui rapidamente, as empresas enfrentam o desafio de integrar novos talentos de maneira eficaz e alinhada à sua cultura. Para solucionar essa dor, as *stakeholders* do projeto, Caroline Vanzellotti e Laura Rucatti, criaram o Decola: uma solução inovadora para essa necessidade, reinventando a maneira como as organizações educam e capacitam seus colaboradores. A plataforma oferece uma metodologia única, combinando ciência de aprendizagem com a essência de cada empresa, garantindo uma integração suave e um desenvolvimento profundo e prático dos novos membros.

Com uma abordagem que valoriza a aprendizagem significativa e o treinamento baseado na prática, o Decola facilita não apenas a rápida adaptação dos colaboradores, mas também fortalece seu senso de pertencimento e engajamento. A utilização de técnicas como *microlearning*, andragogia e aprendizagem mista permite a criação de experiências de aprendizado personalizadas, tornando o processo de *onboarding* mais eficiente e alinhado às expectativas dos talentos e das organizações.

A execução do projeto ocorreu no primeiro semestre de 2024, entre os dias 26 de fevereiro e 26 de junho, pelos estudantes do bacharelado de Engenharia de Software em diversas etapas da AGES, com orientação do professor Edson Moreno. A foto do time responsável pelo projeto pode ser vista na Figura 7.

Figura 7 – Time do projeto Decola



Fonte: Wiki do projeto

3.2 Desenvolvimento do Projeto

Esta seção apresenta informações relativas ao desenvolvimento do projeto: localização do código-fonte e documentação, arquitetura e tecnologias utilizadas, e protótipos de tela desenvolvidos. Toda a documentação do projeto pode ser encontrada na wiki do mesmo: `https://tools.ages.pucrs.br/plataforma-onboarding-para-novos-colaboradores/decola-wiki/-/wikis/home`.

3.2.1 Repositório do Código Fonte do Projeto

Todo o código do projeto encontra-se distribuído em dois repositórios:

- código do *backend*: `https://tools.ages.pucrs.br/plataforma-onboarding-para-novos-colaboradores/decola-api`;
- código do *frontend*: `https://tools.ages.pucrs.br/plataforma-onboarding-para-novos-colaboradores/decola-frontend`.

Há, ainda, um terceiro repositório, usado somente para abrigar a wiki do projeto: `https://tools.ages.pucrs.br/plataforma-onboarding-para-novos-colaboradores/decola-wiki/-/wikis/home`.

3.2.2 Banco de Dados Utilizado

O sistema de gerenciamento de banco de dados escolhido foi o PostgreSQL, um banco de dados relacional de código aberto reconhecido por sua robustez, desem-

penho e extensibilidade. Para a manipulação dos dados, foi utilizado o TypeORM, um Object-Relational Mapping (ORM) (~~*Object Relational Mapper*~~) que mapeia os objetos da aplicação para as entidades do banco, simplificando a comunicação entre a aplicação e o banco de dados relacional.

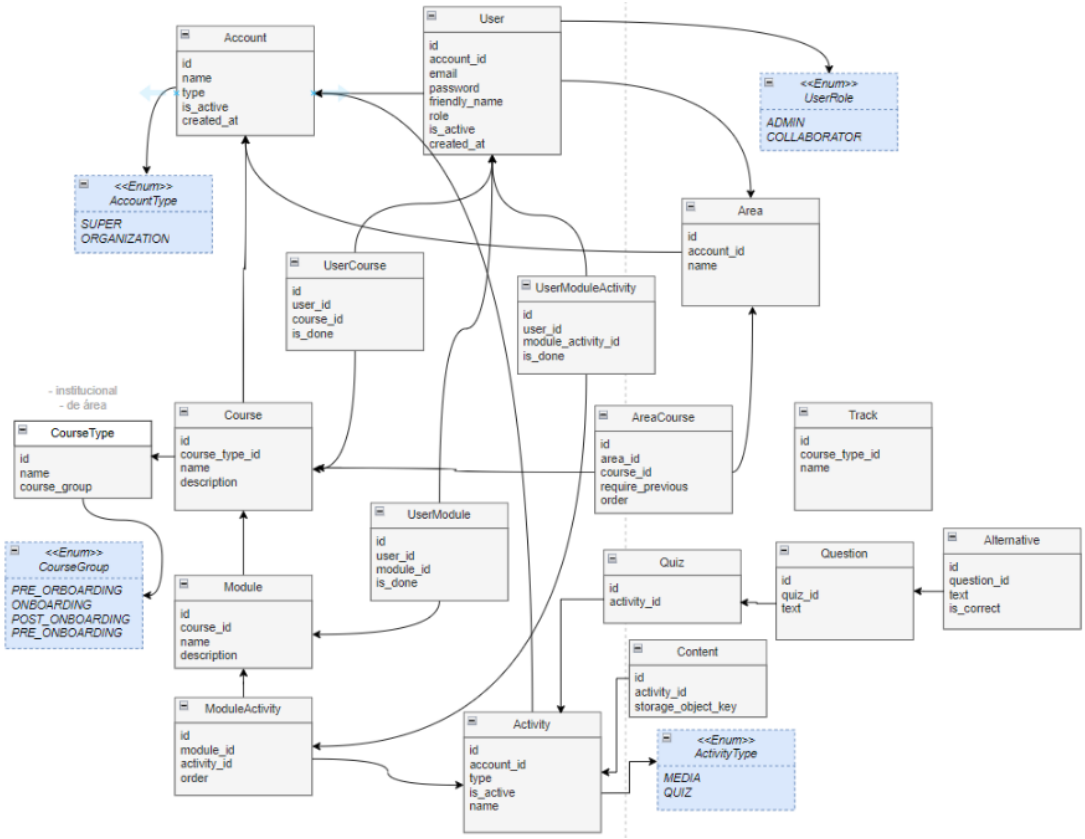
O banco de dados organiza e gerencia as informações de uma plataforma de aprendizado, na qual os usuários podem se inscrever em cursos, concluir módulos e atividades e interagir com conteúdos educativos. Ele armazena informações sobre contas, que podem representar organizações, e os usuários associados a elas, definindo tipos de conta e papéis de usuário, como administradores e colaboradores.

Dentro da plataforma, os cursos são estruturados e categorizados por tipos e grupos. Cada curso pode conter múltiplos módulos, que são unidades menores de aprendizado, e cada módulo inclui atividades que os usuários devem completar, como conteúdo de mídia ou questionários. Os questionários, por sua vez, contêm perguntas com alternativas corretas e incorretas, permitindo a avaliação das respostas dos usuários.

O banco de dados também rastreia o progresso dos usuários em cursos e módulos, registrando as atividades concluídas e permitindo que os administradores acompanhem o desempenho de cada colaborador. Como as contas estão associadas a áreas, diferentes organizações podem personalizar o conteúdo e gerenciar seus próprios usuários e materiais educativos, tornando a plataforma mais interativa e eficiente.

A Figura 8 apresenta o modelo relacional do banco de dados, evidenciando as principais entidades — como *Account*, *User*, *Area*, *Course*, *Module*, *Activity*, *Quiz* e *Question* — e os relacionamentos entre elas.

Figura 8 – Modelo relacional do banco de dados



Fonte: Wiki do projeto

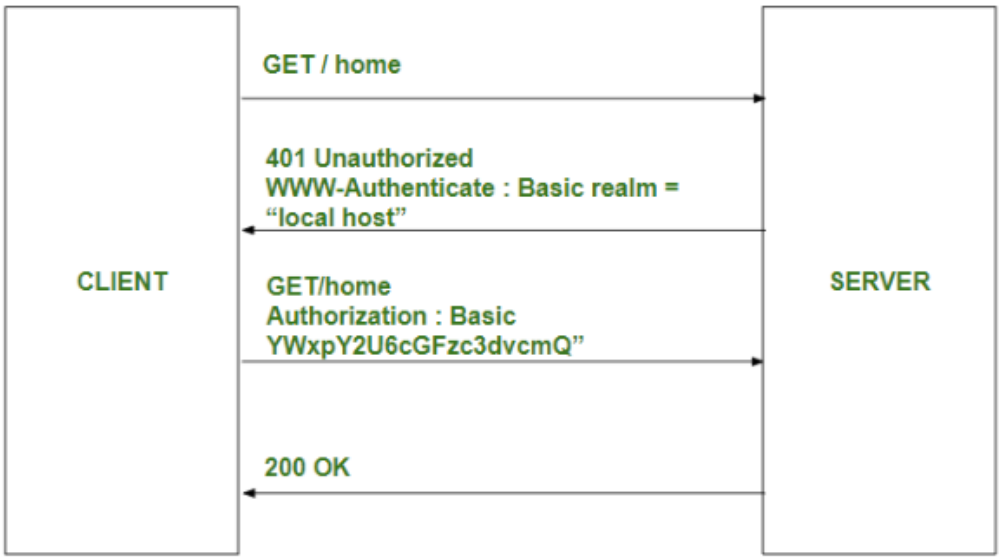
3.2.3 Arquitetura Utilizada

3.2.3.1 Backend

O *backend* do projeto é uma API Representational State Transfer (REST) construída utilizando o NestJS, uma estrutura para criar aplicativos Node.js eficientes e escaláveis do lado do servidor.

Utilizamos JSON Web Token (JWT) para autenticação e autorização. O JWT é um padrão aberto (RFC 7519) para transmitir informações seguras e confiáveis entre partes. Ele consiste em uma cadeia compacta e auto-assinada de informações codificadas em formato JavaScript Object Notation (JSON), que podem ser usadas para autenticação de usuários em aplicativos web e *mobile*, bem como para comunicação entre sistemas. O fluxo de autenticação entre cliente e servidor é ilustrado na Figura 9.

Figura 9 – Modelo de autenticação



Fonte: Wiki do projeto

O fluxo de trabalho do JWT é dividido em três etapas:

- **Geração do Token:** a primeira etapa é a geração do token JWT. Um servidor ou aplicativo gera um token JWT contendo as informações necessárias para autenticar o usuário. O token é geralmente gerado após o usuário fornecer suas credenciais (como nome de usuário e senha) e o servidor verificar essas credenciais. O token é assinado com uma chave secreta conhecida apenas pelo servidor ou aplicativo que o gerou.

- **Envio do Token:** a segunda etapa é o envio do token JWT. O servidor ou aplicativo envia o token JWT para o cliente, geralmente em um cabeçalho Hypertext Transfer Protocol (HTTP) `Authorization`. O cliente armazena o token para uso posterior, como em solicitações subsequentes para o servidor.

- **Verificação do Token:** a terceira etapa é a verificação do token JWT. O servidor ou aplicativo verifica se o token JWT enviado pelo cliente é válido. A verificação da assinatura do token é feita usando a chave secreta. Se a assinatura estiver correta, o servidor extrai as informações do token e verifica se o usuário tem permissão para acessar o recurso solicitado. Se o token for inválido ou expirado, o servidor nega o acesso ao recurso solicitado.

3.2.3.2 Frontend

Nossa arquitetura de *frontend* aplica os conceitos de Domain Driven Development (DDD) e Separation of Concerns (SoC). Com DDD, agrupamos funcionalidades com comportamentos similares e desacoplamos esses grupos ao máximo de outros componentes. Com SoC, nosso objetivo é separar lógica, *views* e modelos de dados, seguindo um padrão semelhante ao MVC.

3.2.4 Protótipos das Telas Desenvolvidas

Os protótipos deste projeto estão disponíveis na Wiki do projeto: `https://tools.ages.pucrs.br/plataforma-onboarding-para-novos-colaboradores/decola-wiki/-/wikis/mockups`.

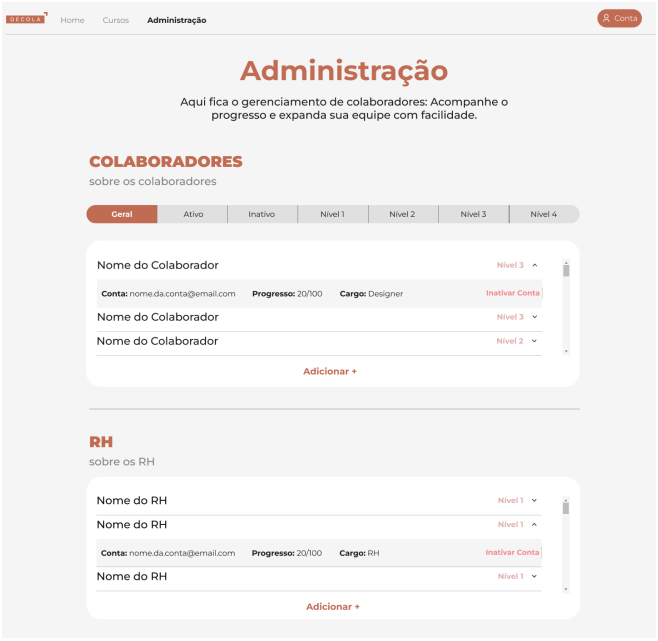
O design da aplicação, como pode ser visto a seguir, poderia ser aprimorado, porém à época era considerado agradável, algo que os *stakeholders* aprovaram bastante. As Figuras 10, 11 e 12 tangem o fluxo do RH, contemplando a página inicial, a administração de colaboradores e contas, e a gestão das áreas e dos níveis de acesso. Já as Figuras 13 e 14 abrangem o fluxo do colaborador, apresentando uma atividade de múltipla escolha e uma atividade de conteúdo externo tal como elas são exibidas ao usuário durante a realização de um módulo.

Figura 10 – Tela de página inicial



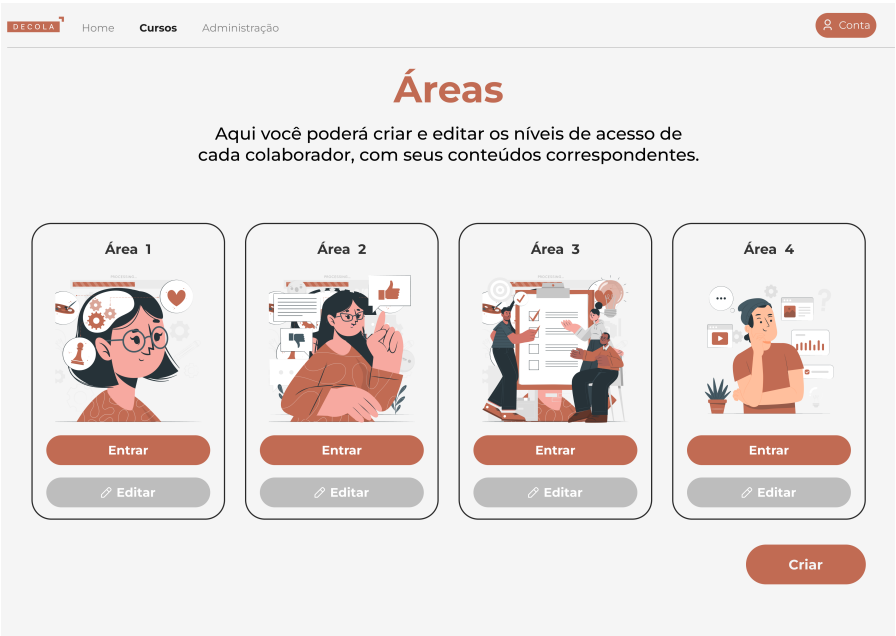
Fonte: Wiki do projeto

Figura 11 – Tela de administração



Fonte: Wiki do projeto

Figura 12 – Tela de áreas



Fonte: Wiki do projeto

Figura 13 – Tela de atividade de múltipla escolha



Fonte: Wiki do projeto

Figura 14 – Tela de atividade de conteúdo externo



Fonte: Wiki do projeto

3.2.5 Tecnologias Utilizadas

- **PostgreSQL** (PostgreSQL Global Development Group, 1996): Sistema de gerenciamento de banco de dados relacional de código aberto, conhecido por sua

robustez, extensibilidade e conformidade com os padrões Structured Query Language (SQL).

- **TypeORM** (TypeORM, 2016): Biblioteca ORM para TypeScript e JavaScript que facilita a interação com bancos de dados relacionais através de um mapeamento objeto-relacional eficiente e intuitivo.
- **AWS** (Amazon, 2006): Plataforma de computação em nuvem fornecida pela Amazon, oferecendo uma ampla gama de serviços, incluindo armazenamento, redes, bancos de dados e ferramentas de desenvolvimento, para escalabilidade e flexibilidade na construção de aplicações.
- **TypeScript** (Microsoft, 2012): Linguagem de programação de código aberto desenvolvida pela Microsoft que adiciona tipagem estática opcional ao JavaScript, melhorando a produtividade e a qualidade do código.
- **NestJS** (NestJS, 2017): *Framework* progressivo para Node.js que permite a construção de aplicativos do lado do servidor escaláveis e eficientes utilizando TypeScript.
- **React** (Meta, 2013): Biblioteca JavaScript de código aberto para a construção de interfaces de usuário, desenvolvida pelo Facebook, amplamente utilizada para criar aplicações web e *mobile* dinâmicas e interativas.
- **Sass** (Sass, 2006): Extensão do CSS que permite o uso de variáveis, aninhamento e *mixins*, facilitando a escrita e manutenção de estilos, tornando o desenvolvimento *front-end* mais eficiente.

3.3 Atividades Desempenhadas Pelo Aluno no Projeto

3.3.1 Sprint 0

Durante a sprint inicial, dediquei minha atenção ao aprendizado das tecnologias de *front-end*. Essa escolha, embora me afastasse das atividades chave desta fase, especificamente a elaboração dos *mockups*, mostrou-se fundamental para adquirir uma visão abrangente sobre a estrutura e os requisitos do projeto. Este período destacou

para mim a importância de uma atitude proativa e de uma busca contínua por maneiras de agregar valor à equipe. Entendi que questionar e solicitar esclarecimentos não só enriquece meu entendimento sobre as tarefas em mãos, mas também incentiva uma cultura de colaboração e partilha de saberes entre os membros da equipe.

Este foco único permitiu-me mergulhar nos detalhes de React e TypeScript, com o objetivo de fortalecer significativamente minha capacidade de contribuir para o projeto nas fases futuras. Assim, este esforço não apenas preparou o terreno para uma participação mais efetiva nas próximas sprints, mas também sublinhou o valor de uma preparação cuidadosa e de uma participação ativa na construção coletiva do projeto.

3.3.2 Sprint 1

Nesta sprint, enfrentamos o desafio de não ter as *stakeholders* presentes na apresentação dos *mockups*. Para contornar isso, os AGES IV do nosso time decidiram organizar uma reunião extraclasse, garantindo assim que estávamos no caminho correto. Embora não tenha conseguido participar dessa reunião, o *feedback* recebido foi bastante positivo.

Durante o planejamento, dividimos o time em duas grandes equipes: *backend* e *frontend*. Essas, por sua vez, se subdividiram em duplas ou trios para atacar as US da sprint. Integrei a equipe de *front-end*, assumindo a responsabilidade pela tela inicial do RH. Dentro dessa tarefa, encarreguei-me especificamente do carrossel da tela e, posteriormente, colaborei com o AGES I responsável por outro componente dessa mesma tela, Andrei Rech.

Ao desenvolver o carrossel, optei inicialmente por utilizar uma biblioteca que já oferecia as funcionalidades necessárias, necessitando apenas de estilização adicional. Contudo, após finalizar essa etapa e discutir com os AGES III, decidimos refatorar o código para eliminar o uso da biblioteca, favorecendo um desenvolvimento completamente nativo. Felizmente, a finalização desse componente foi rápida, graças ao meu prévio estudo das tecnologias envolvidas.

Para compor a tela inicial, eu e o AGES I, Andrei Rech, nos reunimos durante uma aula para realizar os ajustes necessários, visando alcançar a responsividade mínima e alinhar o design com os *mockups* no Figma. Um ponto pendente, no entanto, foi a barra de navegação, a cargo de outra equipe. Apesar disso, deixamos nossa tela prati-

camente pronta, aguardando apenas a integração com o *backend* e a implementação da barra de navegação.

A falta de uma apresentação direta para as *stakeholders* foi um contratempo, mas o progresso alcançado e a qualidade do trabalho entregue pelo time, especialmente na primeira sprint de desenvolvimento, me deixaram particularmente satisfeito.

3.3.3 Sprint 2

Após a apresentação remota feita pelos AGES IV para as *stakeholders*, percebemos que estávamos no caminho certo. Durante a *planning*, minha *squad* ficou responsável pela página de Módulos.

Na mesma aula, juntamente com o AGES I Andrei Rech, desenvolvemos o componente principal da tela, o *card*, que futuramente seria utilizado em outras telas. Ao fim da aula, distribuímos as tarefas para agilizar o processo, ficando eu responsável por toda a parte de ajustar a *Modal* que seria utilizada para os Create, Read, Update, Delete (CRUD).

Estudei para entender como customizar a *Modal* adequadamente. Comecei criando um esqueleto com alguns componentes que já tinham sido desenvolvidos pela equipe na sprint passada. Ajustei detalhes importantes, como a lógica para alterar a imagem do Módulo. Apesar de ter demorado um pouco, consegui fazer com que os dados alterados na *Modal* refletissem corretamente na tela principal.

No início da última semana, minha *squad* conseguiu finalizar as tarefas atribuídas. Decidimos então começar a desenvolver a tela de Cursos, que, no fim, já estava quase pronta, pois era muito semelhante à de Módulos.

No final desta sprint, as enchentes que assolaram grande parte do estado do Rio Grande do Sul afetaram nossa entrega. O prazo foi postergado até que fosse estabelecido um retorno, inicialmente remoto, para que pudéssemos apresentar.

No dia da apresentação, apenas uma das *stakeholders* compareceu, o que desanimou um pouco a equipe. Após tanto esforço e tempo dedicados, a ausência das demais *stakeholders* fez parecer que não estavam motivadas para ver o progresso do projeto.

3.3.4 Sprint 3

Ainda remoto, nossa Sprint 3 foi realizada durante toda a sua duração. Após a reunião com as *stakeholders*, fizemos nossa *planning* e foi decidido previamente que esta sprint, para o time de *frontend*, seria basicamente para fazer a integração de todas as páginas e funcionalidades que tínhamos desenvolvido, com o *backend*.

Minha *squad* novamente ficou responsável por fazer a integração de módulos e cursos. Começamos pelos módulos, e tivemos um pouco de dificuldade no início para entender como deveriam ser feitas as requisições, que antes estavam *mockadas*. Um problema que encontramos foram as mensagens de erros muito genéricas, o que não ajudou a identificar os problemas, exigindo testes extensivos para descobrir onde estavam os erros. Além disso, o Swagger apresentava muitas inconsistências entre o que era necessário ser passado e os exemplos fornecidos. Após várias tentativas, conseguimos fazer uma integração parcial de módulos e cursos.

Como o *backend* não havia desenvolvido o *endpoint* para que fosse possível o envio e requisição de imagens, isso ficou como débito técnico para a última sprint. Durante esta sprint, também criei o componente para utilizar na marcação de progresso do usuário conforme ele fosse concluindo os módulos.

Durante a apresentação, novamente apenas uma *stakeholder* apareceu e, mais uma vez, não apontou nenhum ponto de melhoria ou algo que gostaria de ver diferente. Isso deixou o time frustrado, pois parecia que não estavam se importando com o nosso trabalho. Sinto também que, nesta sprint, não consegui me dedicar tanto ao projeto quanto gostaria.

3.3.5 Sprint 4

Com a retomada ainda precoce das aulas presenciais, o time se reuniu para a *planning* para separar as últimas *tasks*. Durante o meio tempo entre a apresentação e a *planning*, um dos nossos AGES III fez um *rebase* no *backend* do projeto para permitir que as imagens pudessem ser salvas e também para melhorar a documentação do Swagger. Por conta disso, tivemos que refatorar a integração de cursos e módulos. Anteriormente, estávamos salvando apenas a Uniform Resource Locator (URL) da imagem, mas isso estava causando erros na integração com a Amazon Web Services (AWS), o que tornou a refatoração necessária.

Dividimos essa tarefa em duas etapas. Uma parte ficou responsável pelo código que exibe as imagens na tela, enquanto a outra se encarregou de enviar a imagem selecionada pelo usuário para o banco de dados. Fiquei responsável por esta última parte.

Na sprint anterior, aprendi muito sobre *mutations* e *queries* no React, o que facilitou bastante a execução da minha tarefa. Assim que ambas as partes estavam prontas, nos reunimos para ajustar os detalhes necessários e garantir que a entrega ficasse o melhor possível.

Ao contrário da sprint anterior, consegui entregar tudo que me foi proposto de forma satisfatória. Mesmo com todas as dificuldades enfrentadas devido às aulas presenciais e o estado de calamidade pública, sinto que entreguei o melhor que pude no momento. Durante a apresentação para as *stakeholders*, novamente tivemos a presença apenas de uma delas, que aprovou o projeto como um todo e ficou super satisfeita com o nosso trabalho.

3.4 Conclusão

A tragédia que assolou o Rio Grande do Sul impactou diretamente o fluxo de trabalho, engajamento e andamento do projeto. Além disso, a falta de engajamento e de críticas construtivas por parte das *stakeholders* foi desmotivadora, especialmente quando apenas uma delas comparecia às apresentações e não fornecia nenhum tipo de *feedback*.

No entanto, a experiência de desenvolver um projeto com entregas iterativas e incrementais, com o compromisso de entregar um sistema funcional ao final, foi muito valiosa. A cada sprint, enfrentamos desafios como a integração das funcionalidades e a refatoração necessária para resolver problemas técnicos, o que nos permitiu aprimorar nossas habilidades.

Enfrentamos alguns problemas de comunicação dentro da equipe, resultando em atrasos na entrega de algumas funcionalidades. O principal obstáculo foi a necessidade de refatorar a integração de imagens devido a mudanças no *backend* e dificuldades com a AWS. Apesar desses desafios, conseguimos entregar um produto funcional. Sinto que poderíamos ter alcançado um resultado ainda melhor se não fossem os imprevistos e a falta de engajamento das *stakeholders*.

No fim deste projeto, me sinto um pouco frustrado, pois acredito que poderíamos ter entregue um projeto muito melhor caso não houvesse ocorrido esses pontos críticos. No entanto, este semestre me proporcionou um crescimento pessoal único. Saber lidar com esses tipos de obstáculos durante o desenvolvimento de software é um aprendizado incrível para que eu possa melhorar minhas habilidades técnicas e de resolução de problemas.

No que se refere à relação entre as disciplinas cursadas e a AGES, acredito que as que mais me ajudaram a enfrentar os desafios do projeto Decola foram Modelagem e Projeto de Banco de Dados e Design de Interação. A primeira me deu a base necessária para auxiliar na modelagem das primeiras versões do banco de dados da aplicação, contribuindo para a definição das entidades e dos relacionamentos. Já a segunda foi especialmente útil porque atuei mais na parte do *frontend* neste projeto, ajudando-me a compreender o objetivo de um design bem feito e organizado, bem como a importância de uma boa experiência para o usuário.

De forma geral, foi muito satisfatório participar do desenvolvimento da aplicação Decola e espero que o trabalho desenvolvido na AGES seja continuado pelas clientes.

4 AGES III — TOTEM INTERATIVO

Esta seção busca apresentar a passagem do aluno como AGES III pelo Projeto Totem Interativo. Aqui estão descritos os artefatos entregues, a atuação ao longo das sprints e os pontos de melhoria identificados.

4.1 Introdução

Em ambientes dinâmicos como feiras setoriais, visitantes e expositores enfrentam frequentemente dificuldades relacionadas à localização precisa dos fornecedores e ao registro eficiente de contatos, fatores críticos para o sucesso desses eventos. Atento a essa necessidade, o *stakeholder* Igor Daudt idealizou o desenvolvimento de um totem interativo, projetado para superar esses desafios. O sistema proposto visa facilitar a interação dos visitantes com expositores através de um mapa tridimensional imersivo, tornando a experiência mais intuitiva e prática.

Com uma aplicação *off-line touch-screen*, o totem permitirá que usuários busquem facilmente produtos e fornecedores específicos em um ambiente visualmente atraente e interativo. O registro automatizado de visitantes por meio da leitura dos crachás otimizará significativamente a captura de contatos, e posteriormente relatórios detalhados serão entregues aos expositores, oferecendo *insights* valiosos.

A execução do projeto ocorreu no primeiro semestre de 2025, entre os dias 28 de fevereiro e 27 de junho de 2025, compreendendo diversas etapas realizadas pelos estudantes do bacharelado em Engenharia de Software no âmbito da AGES, sob a orientação do professor ~~Dr.~~ Azriel Majdenbaum. A foto do time responsável pelo projeto pode ser vista na Figura 15.

Figura 15 – Time do projeto Totem Interativo



Fonte: Wiki do projeto

4.2 Desenvolvimento do Projeto

Esta seção apresenta informações relativas ao desenvolvimento do projeto: localização do código-fonte e documentação, arquitetura e tecnologias utilizadas, e protótipos de tela desenvolvidos. Toda a documentação do projeto pode ser encontrada na wiki do mesmo: `https://tools.ages.pucrs.br/totem-interativo/wiki/-/wikis/home`.

4.2.1 Repositório do Código Fonte do Projeto

Todo o código do projeto encontra-se distribuído em dois repositórios:

- código do *backend*: `https://tools.ages.pucrs.br/totem-interativo/backend`;
- código do *frontend*: `https://tools.ages.pucrs.br/totem-interativo/frontend`.

Há, ainda, um terceiro repositório, usado somente para abrigar a wiki do projeto: `https://tools.ages.pucrs.br/totem-interativo/wiki/-/wikis/home`.

4.2.2 Banco de Dados Utilizado

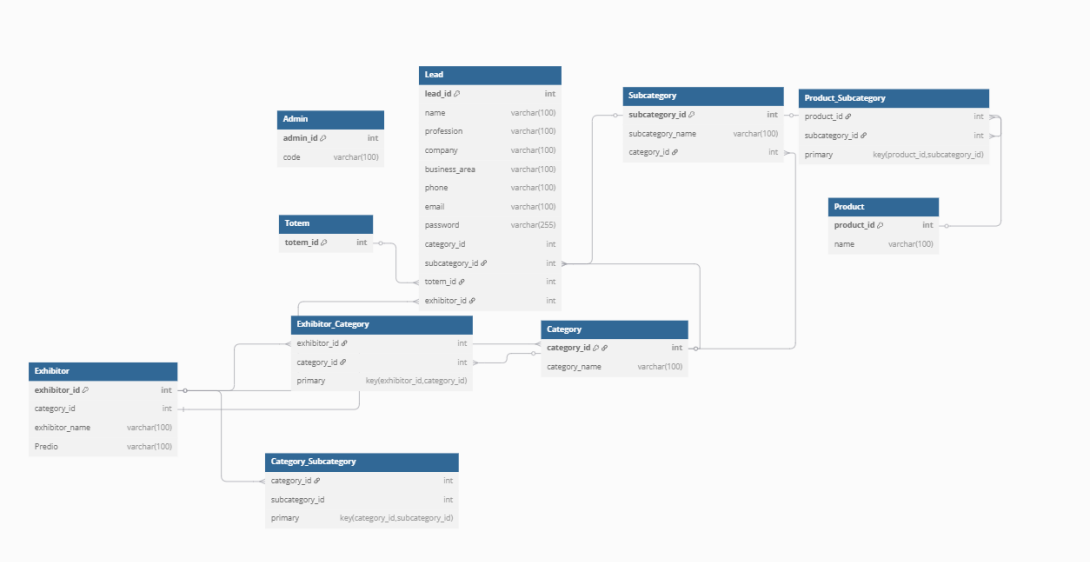
Para o desenvolvimento do projeto, optamos pelo SQLite como sistema de gerenciamento de banco de dados. Essa escolha foi estratégica, considerando dois fatores críticos: a necessidade de operação *offline* e as limitações de *hardware* dos totens. O SQLite, sendo extremamente leve e eficiente, consome recursos mínimos de memória e processamento, ideal para dispositivos com capacidade limitada.

O banco foi estruturado para gerenciar informações de feiras e eventos de forma otimizada. A arquitetura prioriza consultas rápidas e diretas, organizando dados de visitantes (*leads*), expositores, produtos e categorias para minimizar operações complexas.

A tabela *Lead* constitui o núcleo do sistema, armazenando dados completos dos visitantes — informações pessoais, profissionais e suas interações com o totem. Cada *lead* é vinculado às categorias de interesse e expositores pesquisados, criando um perfil detalhado de comportamento.

Os expositores são organizados com suporte a múltiplas categorias, incluindo o atributo “Predio” — adição fundamental para o caso de uso do Tecnopuc, permitindo localização física precisa dentro do parque tecnológico. O sistema de categorização hierárquico (categorias e subcategorias) foi implementado priorizando *performance*, com índices estratégicos para acelerar as buscas mais frequentes. A Figura 16 apresenta o modelo relacional do banco de dados.

Figura 16 – Modelo relacional do banco de dados



Fonte: Documentação do projeto

A escolha do SQLite provou-se acertada para o contexto do projeto. Sua natureza *serverless* elimina *overhead* de comunicação, seu *footprint* mínimo preserva recursos do sistema, e sua capacidade de operar em memória garante respostas rápidas. A estrutura foi projetada evitando junções múltiplas e subconsultas complexas, garantindo fluidez mesmo em *hardware* básico. Essa combinação de banco leve com modelagem eficiente demonstrou que é possível criar sistemas robustos operando sob severas restrições de *hardware*, proporcionando experiência responsiva aos usuários das feiras.

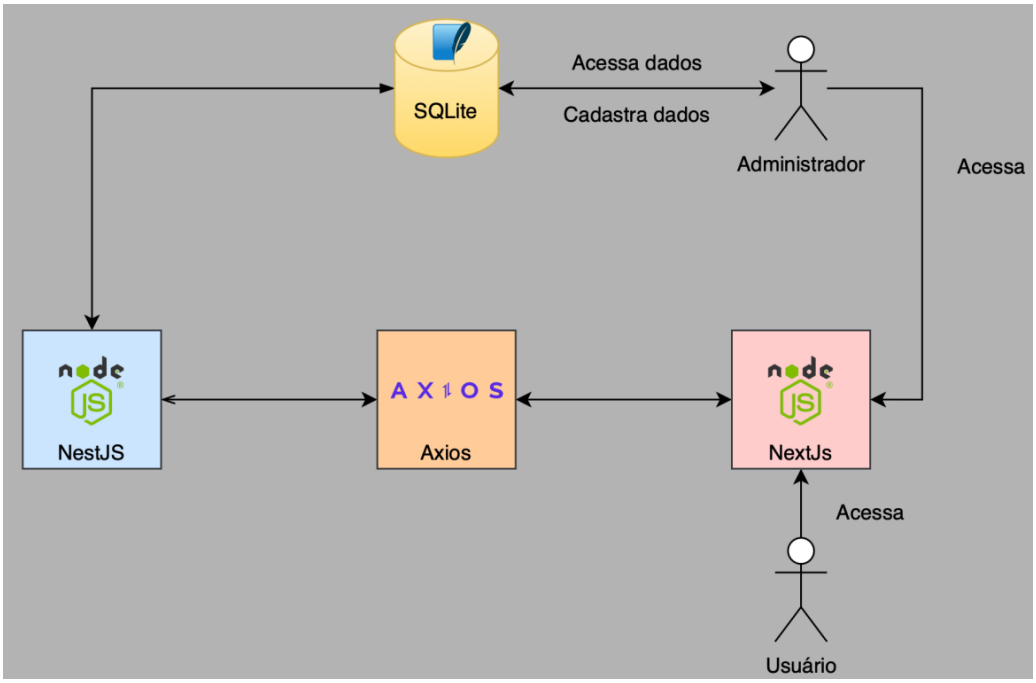
4.2.3 Arquitetura Utilizada

O Totem Interativo adota uma arquitetura cliente-servidor, com separação explícita entre o *frontend* — a aplicação que roda no próprio totem — e o *backend*, responsável pela lógica de negócio e pelo acesso aos dados. Os dois lados comunicam-se exclusivamente por uma API REST sobre HTTP, o que permitiu desenvolver as camadas em paralelo, isolar responsabilidades e evoluir cada parte de forma independente.

Essa separação atende a uma restrição central do projeto: como o totem precisa operar mesmo sem conexão externa, o *frontend*, o *backend* e o banco de dados são executados localmente no próprio equipamento, e a API atua como a única fronteira entre a interface e os dados. A Figura 17 ilustra esse fluxo: a interface em Next.js consome a API em NestJS por meio do cliente HTTP Axios, que por sua vez acessa

o banco SQLite. O diagrama também distingue os dois perfis de uso — o visitante, que consulta as informações no totem, e o administrador, que cadastra e gerencia os dados. As subseções seguintes detalham cada lado.

Figura 17 – Arquitetura do Totem Interativo e fluxo de comunicação do *frontend*



Fonte: Wiki do projeto

4.2.3.1 Frontend

A arquitetura do *frontend* foi concebida para oferecer uma aplicação interativa com renderização 3D. A estrutura modular facilitou tanto o desenvolvimento quanto a manutenção.

Os componentes da interface foram organizados em grupos lógicos:

- *dropdown*: para menus de seleção e filtros;
- *map*: responsável pela renderização e interação com o mapa 3D;
- *searchbar*: para funcionalidades de busca;
- *sidebar*: para navegação e exibição de informações contextuais.

A comunicação com o *backend* foi centralizada na camada de *services*, atuando como um ponto único de integração com as APIs. O gerenciamento de estados globais

é feito via *contexts*, e lógicas reutilizáveis, como o *logout* automático por inatividade, foram isoladas em *hooks*.

Recursos estáticos essenciais para o mapa 3D foram estrategicamente organizados:

- *models*: arquivos de geometria dos edifícios;
- *textures*: materiais visuais aplicados aos modelos;
- *fonts*: tipografias personalizadas para a identidade visual;
- *assets*: recursos gráficos gerais da aplicação.

As rotas da aplicação são diferenciadas conforme o perfil do usuário: visitantes acessam a interface principal do totem, enquanto administradores dispõem de páginas específicas para importação de dados (*admin-import*) e exportação de relatórios (*admin*). A funcionalidade de *QR Code* possui um módulo próprio com componentes dedicados.

4.2.3.2 Backend

A arquitetura do *backend* adotou padrões de desenvolvimento em camadas, garantindo uma clara segregação de responsabilidades e facilitando os processos de teste e manutenção.

A estrutura de camadas implementada é composta por:

- *Controllers*: atuam como ponto de entrada para as requisições HTTP;
- *Services*: encapsulam toda a lógica de negócio da aplicação;
- *DTOs*: definem os contratos de dados para entrada e saída;
- *Entities*: representam as estruturas do banco de dados;
- *Modules*: agrupam logicamente as funcionalidades.

Aspectos transversais foram concentrados no diretório *common*:

- *Guards*: para proteção de rotas administrativas;
- *Filters*: para tratamento padronizado de erros;

- *Mapper*: para transformações entre diferentes representações de dados.

O armazenamento local foi estruturado no diretório *database*, contendo o arquivo principal SQLite.

O fluxo de dados segue um padrão consistente: as requisições chegam aos *controllers*, são validadas pelos *DTOs*, processadas pelos *services* (que interagem com as *entities*), e então retornam respostas estruturadas. As operações administrativas incluem *endpoints* especializados para importação em massa via Comma-Separated Values (CSV) e exportação de dados agregados.

A estrutura de testes replica a organização principal, com diretórios específicos para *controllers* e *services*, assegurando uma cobertura adequada das funcionalidades críticas.

4.2.4 Protótipos das Telas Desenvolvidas

Os protótipos deste projeto estão disponíveis na wiki do projeto: `https://tools.ages.pucrs.br/totem-interativo/wiki/-/wikis/mockups`.

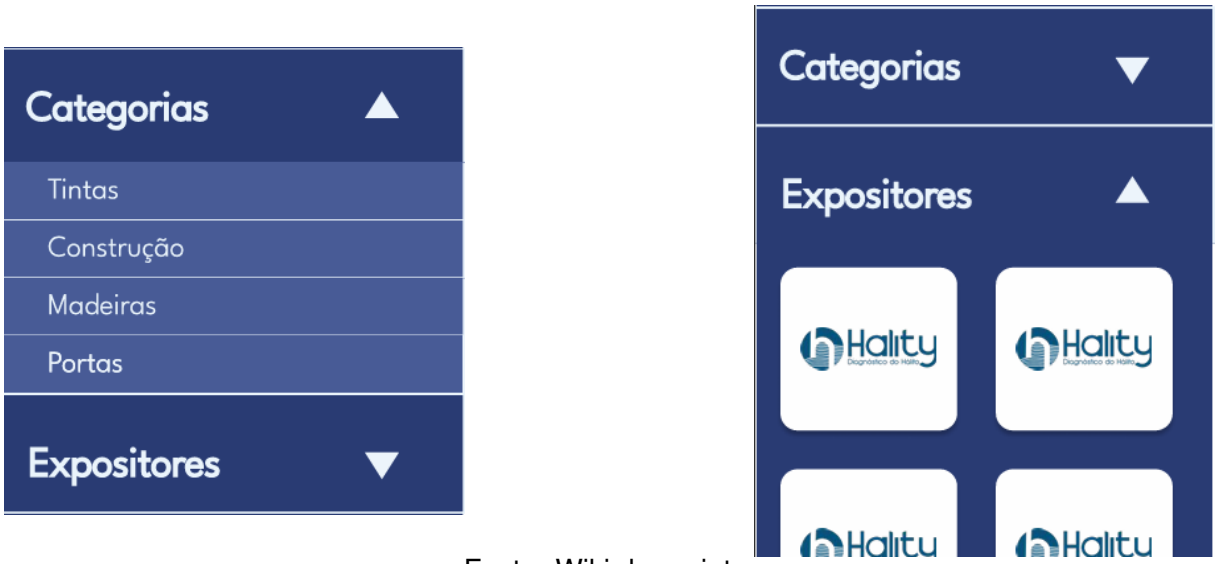
O design da aplicação, como pode ser visto a seguir, poderia ser aprimorado, porém à época era considerado agradável, algo que o cliente aprovou bastante. As Figuras 18, 19 e 20 tangem o fluxo do visitante, contemplando a tela inicial com o mapa interativo, os menus de categorias e expositores, e a busca de expositores no totem. Já a Figura 21 abrange o fluxo do administrador, responsável pela importação dos dados e pela configuração do evento.

Figura 18 – Tela inicial com mapa interativo



Fonte: Wiki do projeto

Figura 19 – Menus de categorias e expositores



Fonte: Wiki do projeto

Figura 20 – Busca de expositores



Fonte: Wiki do projeto

Figura 21 – Tela de configurações do administrador



Fonte: Wiki do projeto

4.2.5 Tecnologias Utilizadas

Nesta seção serão apresentadas as principais tecnologias empregadas no desenvolvimento do projeto Totem Interativo, destacando suas contribuições para o sucesso da implementação.

- **React** (Meta, 2013): Biblioteca JavaScript de código aberto para construção de interfaces de usuário reativas e componentes reutilizáveis, fundamental para criar a interface interativa do totem com atualizações dinâmicas.
- **Three.js** (Three.js, 2010): Biblioteca JavaScript para renderização 3D no navegador, responsável pela criação e manipulação do mapa tridimensional interativo, permitindo visualização imersiva dos espaços e interações com elementos 3D.
- **Next.js** (Vercel, 2016): *Framework* React para desenvolvimento *web* com renderização híbrida e otimizações automáticas de *performance*, garantindo resposta rápida mesmo em *hardware* limitado.
- **Node.js** (Node.js Foundation, 2009): Ambiente de execução JavaScript *server-side* com natureza assíncrona e eficiência no gerenciamento de I/O, ideal para múltiplas requisições simultâneas.

- **NestJS** (NestJS, 2017): *Framework* progressivo para Node.js que permite construção de aplicativos escaláveis com arquitetura modular e suporte nativo para TypeScript.
- **TypeScript** (Microsoft, 2012): *Superset* do JavaScript que adiciona tipagem estática opcional, reduzindo *bugs* em tempo de desenvolvimento e facilitando manutenção do código.
- **Swagger** (OpenAPI Initiative, 2011): Ferramenta para documentação e testes interativos de APIs RESTful, integrada ao NestJS para gerar documentação automática dos *endpoints*.
- **SQLite** (SQLite, 2000): Sistema de banco de dados relacional *serverless* e baseado em arquivo único, escolhido por sua leveza e capacidade de operar sem dependências externas em ambiente *offline*.

4.3 Atividades Desempenhadas Pelo Aluno no Projeto

Nesta seção será abordada a minha contribuição para o projeto durante as Sprints 0, 1, 2, 3 e 4.

4.3.1 Sprint 0

Como AGES III, reconheço que as *sprints* iniciais são fundamentais, pois nelas definimos as tecnologias que serão utilizadas ao longo de todo o projeto. Em conjunto com os demais integrantes da equipe de AGES III, analisamos as necessidades específicas da solução — que envolvia a construção de um mapa 3D interativo — e concluímos que a melhor abordagem seria utilizar uma biblioteca especializada em renderização tridimensional no *frontend*.

Dessa forma, definimos a *stack* tecnológica da seguinte maneira: React seria utilizado como base da aplicação, enquanto a biblioteca Three.js ficaria responsável pela renderização e manipulação do mapa 3D. Para a estilização dos componentes, optamos pelo uso de *styled-components*, aproveitando a flexibilidade da abordagem CSS-in-JS. No *backend*, como nenhum integrante do grupo possuía experiência avançada, optamos por uma *stack* que já era familiar para a maioria e que utilizasse a mesma linguagem do *frontend* — por isso, escolhemos o NestJS, *framework* robusto e bem

estruturado em TypeScript, similar ao *frontend*, garantindo que a curva de aprendizado fosse a menor possível.

Com as definições tecnológicas finalizadas, iniciamos a criação dos repositórios do projeto. No entanto, enfrentamos um contratempo significativo: o arquiteto responsável pela AGES precisou se desligar da agência, o que gerou atrasos na criação e estruturação inicial dos repositórios e causou certa apreensão na equipe. Ainda assim, conseguimos avançar e estruturar o ambiente inicial de desenvolvimento de forma colaborativa.

Com o tempo restante após todas essas definições, concentrei meus esforços em aprofundar o conhecimento na biblioteca Three.js, identificada desde o início como um dos pontos mais críticos do projeto. Essa dedicação inicial foi essencial para garantir segurança técnica na construção do mapa 3D nas etapas seguintes.

4.3.2 Sprint 1

Na Sprint 1, o time foi dividido em *squads* de quatro pessoas. No entanto, essa divisão não foi feita de maneira estratégica, pois as *tasks* já haviam sido atribuídas previamente. Com isso, as *squads* acabaram tendo um papel meramente protocolar. Como eu já havia me aprofundado bastante no estudo da biblioteca Three.js, assumi a *task* de criar o mapa para a tela do visitante. Estava confiante de que conseguiria entregar um resultado sólido, e mesmo que fosse um mapa estático e não dinâmico, já representaria um bom avanço para esta etapa do projeto.

Durante as aulas, envolvi-me bastante com a equipe que pesquisava formas de automatizar a geração de mapas a partir de arquivos DWG ou DXF. Infelizmente, até a data da apresentação para o *stakeholder*, não conseguimos evoluir significativamente nessa frente.

Com uma semana de antecedência em relação à apresentação, comunicamos a todos que o *code freeze* seria realizado dois dias antes da aula, a fim de garantir tempo hábil para revisar cuidadosamente cada *pull request* e resolver possíveis conflitos de forma organizada.

Concluí minha *task* com certa antecedência e, ao verificar o andamento do *backend*, percebi que nenhum outro AGES III havia contribuído com o repositório até então. O código encontrava-se desorganizado e sem uma estrutura adequada, mas,

por estarmos muito avançados na *sprint*, não havia mais tempo hábil para reestruturar a arquitetura.

Na véspera do *code freeze*, auxiliei o AGES II Kauã Rodrigues Souza com dúvidas relativas à sua *task* e aos critérios de entrega esperados. Percebemos que havia ruído na comunicação entre os AGES III e que não havíamos alinhado previamente como seriam feitos os *pull requests*. Diante do risco que isso representava para a entrega, tomei a iniciativa de revisar e aprovar todos os PRs do repositório do *backend* sozinho. Apesar da responsabilidade, o processo ocorreu de forma tranquila, e fiquei satisfeito com a qualidade do código entregue por aqueles que contribuíram.

Entregamos ao *stakeholder* praticamente todos os itens planejados para a *sprint*, com exceção do componente de barra de pesquisa no *frontend*. Durante a retrospectiva, discutimos pontos de melhoria e definimos ações concretas para evitar os problemas enfrentados nesta etapa. De forma geral, fiquei satisfeito com a entrega, embora tenha ficado com certa desconfiança quanto ao nível de comprometimento de alguns integrantes do time.

4.3.3 Sprint 2

No início desta *sprint*, identifiquei a necessidade de refatorar a estrutura do *backend* para seguir as boas práticas de desenvolvimento. Solicitei à AGES II Luana Thurow que realizasse essa refatoração, incluindo a padronização de nomenclaturas de variáveis, organização de pastas e estrutura de arquivos. O trabalho realizado foi excelente, resultando em um repositório bem organizado que facilitou significativamente o desenvolvimento subsequente.

Minha principal responsabilidade nesta *sprint* era testar o arquivo que o *stakeholder* enviaria, para validar a viabilidade de continuar com um projeto genérico e escalável ou focar em um mapa fixo como prova de conceito. Apesar de ter solicitado o arquivo aos AGES IV em diversas ocasiões, o *stakeholder* não o enviou a tempo de realizarmos os testes necessários.

Enquanto aguardava o arquivo, acompanhei de perto o desenvolvimento das tarefas dos AGES II. Conseguimos implementar logo no início da *sprint* a funcionalidade da barra de pesquisa que havia ficado pendente, permitindo que a equipe focasse em tarefas mais complexas. Foi definido que dois AGES II trabalhariam em conjunto

no desenvolvimento de um *endpoint* para popular o banco de dados através de planilhas CSV — funcionalidade fundamental para garantir a escalabilidade do projeto para diferentes feiras.

Durante as aulas, em conjunto com os outros AGES III, confirmamos que, considerando que o projeto rodaria em *hardware* sem acesso à internet durante sua execução, o uso da AWS não seria necessário e poderia até complicar o fluxo de desenvolvimento, especialmente considerando o tamanho reduzido da equipe de AGES I e II.

No *code freeze*, ao revisar o código da funcionalidade de importação CSV, identifiquei um problema crítico: a implementação esperava uma planilha contendo IDs e relacionamentos nas colunas, o que seria inviável para o uso pretendido. Após conversar com os AGES II responsáveis, descobri que a descrição da tarefa no quadro estava mal formulada e continha informações incorretas. Comuniquei à equipe que esta funcionalidade entraria como débito técnico e que precisaríamos repensar sua implementação.

A apresentação para o *stakeholder* foi desafiadora, pois, sem avanços significativos no *frontend*, os AGES IV demonstraram apenas as implementações dos *endpoints* através do Postman, o que deixou o Igor visivelmente desapontado. Durante a retrospectiva, discutimos a importância de descrições claras e precisas nas tarefas do quadro, e os AGES IV se comprometeram a melhorar esse aspecto para as próximas *sprints*.

Embora não tenha contribuído diretamente com desenvolvimento de código nesta *sprint*, mantive-me engajado no acompanhamento das tarefas do *backend*. Como único AGES III atuando nesta frente, senti a responsabilidade pelos resultados, ficando frustrado por não ter conseguido atender plenamente às expectativas estabelecidas para esta etapa do projeto.

4.3.4 Sprint 3

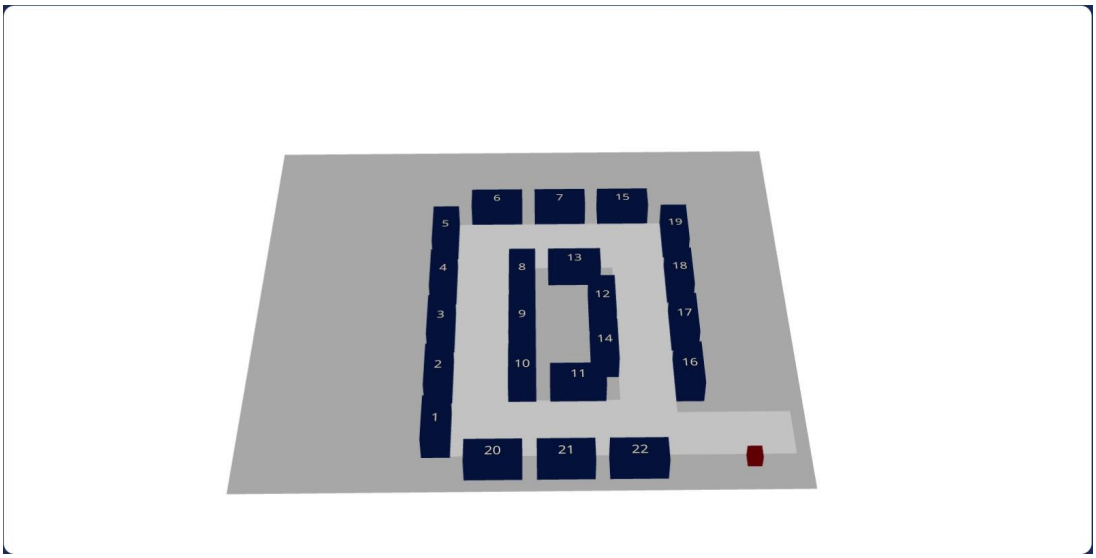
No início desta *sprint*, tomamos a decisão estratégica de focar em um único mapa, mais complexo que a versão inicial, mas ainda relativamente simples para garantir o progresso contínuo do projeto. Imediatamente percebi uma melhora significativa na comunicação dos AGES IV, bem como na qualidade das descrições das tarefas no quadro, o que facilitou consideravelmente o alinhamento entre as expectativas e as

implementações necessárias.

Durante a primeira aula, realizamos uma sessão de *brainstorming* para definir a implementação da funcionalidade de popular o banco de dados, determinando também o formato adequado do arquivo CSV esperado pelo *endpoint*. Após discussões produtivas, chegamos a uma solução elegante e funcional: o CSV deveria conter uma linha para cada produto, simplificando o processo de importação e manutenção dos dados.

Minha responsabilidade principal nesta *sprint* foi desenvolver uma segunda versão do mapa, incluindo mais estandes e um indicador visual da localização do totem. Graças ao estudo aprofundado que havia realizado sobre a biblioteca Three.js, consegui criar rapidamente uma versão aprimorada que atendeu às expectativas como protótipo funcional. O resultado é apresentado na Figura 22.

Figura 22 – Mapa desenvolvido



Fonte: Código *frontend* do projeto

Com o mapa desenvolvido, direcionei meus esforços para realizar a integração com o *backend*. Surpreendentemente, a tarefa foi menos complexa do que havia antecipado, pois o repositório do *frontend* já estava adequadamente estruturado para suportar a integração. Foram necessárias apenas algumas modificações pontuais para garantir uma integração completa e funcional.

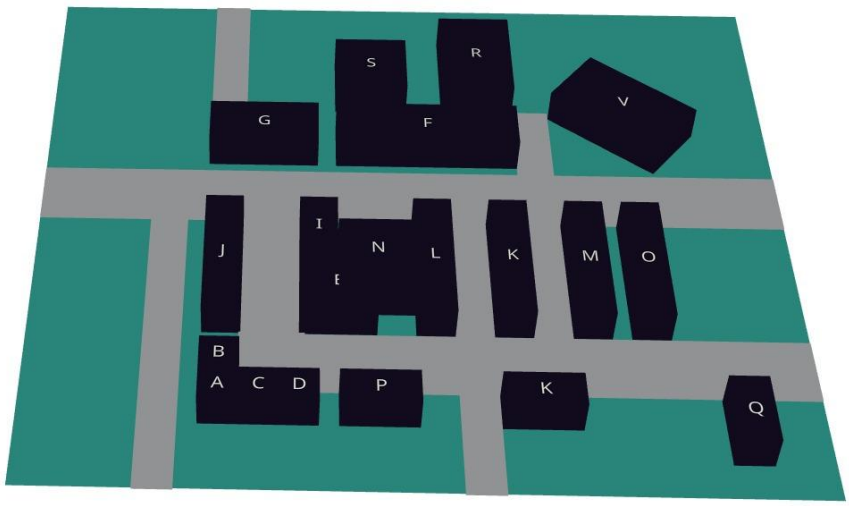
Concluída minha implementação principal, voltei a apoiar o desenvolvimento do *backend* junto aos AGES II. Os dois responsáveis pela funcionalidade de popular o banco estavam com bom progresso, enquanto a terceira AGES II aperfeiçoava a es-

trutura do *backend*, incluindo a documentação Swagger nos *controllers*, trabalho que também estava bem encaminhado.

Durante uma aula, em conjunto com os AGES IV, decidimos explorar uma abordagem inovadora para o mapa. O *stakeholder* havia mencionado no início do semestre sua visão de utilizar o sistema no Tecnopuc, onde a multiplicidade de prédios e empresas torna desafiadora a localização para visitantes externos. Motivados por essa ideia e buscando compensar a entrega insatisfatória da *sprint* anterior, decidimos replicar o mapa do Tecnopuc como uma funcionalidade surpresa.

Aplicando o conhecimento adquirido, consegui implementar o mapa do Tecnopuc sem grandes dificuldades, obtendo aprovação imediata dos AGES IV quanto à qualidade e fidelidade da representação. A Figura 23 apresenta essa primeira versão do mapa do Tecnopuc.

Figura 23 – Primeira versão do mapa do Tecnopuc



Fonte: Repositório *frontend* do projeto

Com minhas tarefas concluídas, dediquei-me à revisão dos *pull requests* dos AGES II durante a fase de *code freeze*. Não identificamos problemas significativos no código do *backend*. Contudo, uma funcionalidade delegada a outro AGES III e um AGES I não foi finalizada, resultando em débito técnico — felizmente, algo que passaria despercebido aos olhos do *stakeholder*.

Durante a última semana e meia da *sprint*, enfrentamos sérios problemas com o ambiente GitLab, incluindo indisponibilidade, lentidão extrema e um *rollback* crítico poucos dias antes do *code freeze*. Embora o projeto não tenha sido severamente afetado, a situação gerou apreensão na equipe sobre possíveis recorrências.

A apresentação desta *sprint* foi um sucesso completo, contrastando drasticamente com a anterior. O *stakeholder* demonstrou extrema satisfação com o progresso, afirmando que apenas com o desenvolvimento realizado até aquele momento, a ideia do projeto já estava validada. Estrategicamente, não havíamos revelado o desenvolvimento do mapa do Tecnopuc. Quando o *stakeholder* sugeriu focar a última *sprint* em um caso de uso específico para o Tecnopuc, os AGES IV apresentaram nossa implementação surpresa. Sua reação foi de genuíno contentamento, prontificando-se imediatamente a fornecer qualquer informação adicional necessária sobre o local.

Esta *sprint* representou um ponto de virada no projeto, não apenas pelos avanços técnicos significativos, mas também pela validação do conceito pelo *stakeholder* e pelo restabelecimento da confiança na capacidade de entrega da equipe.

É importante destacar que, durante esta *sprint*, o AGES III Eduardo Felber Eichner e eu acabamos assumindo responsabilidades adicionais para garantir a continuidade do projeto. Estabelecemos uma divisão clara de trabalho, onde Eduardo concentrou-se exclusivamente no repositório *frontend* enquanto eu gerenciava o *backend* e desenvolvia os mapas. Embora essa abordagem tenha sido eficaz e garantido o sucesso das entregas, reconheço que deveria ter comunicado mais claramente minhas preocupações sobre a distribuição desigual de responsabilidades entre todos os AGES III.

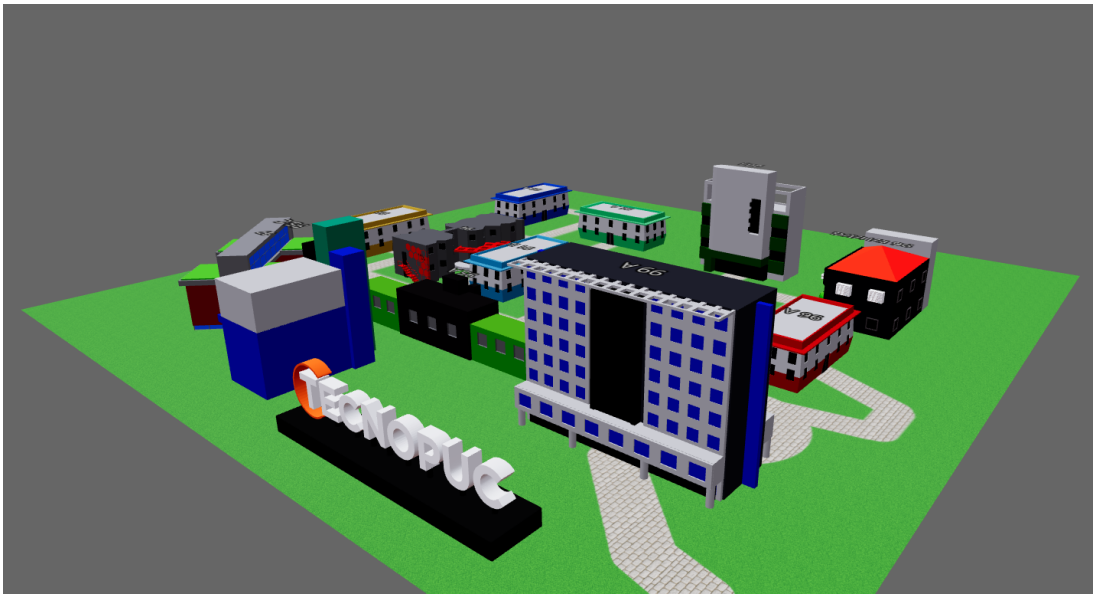
4.3.5 Sprint 4

Nesta *sprint* final, assumi a responsabilidade de aprimorar o mapa com foco específico no Tecnopuc. O AGES IV Alessandro Borges de Souza estabeleceu contato direto com a equipe do parque tecnológico e, surpreendentemente, descobrimos que eles estavam iniciando um projeto com objetivos similares ao nosso. Essa coincidência abriu portas para uma colaboração: a equipe do Tecnopuc se comprometeu a fornecer uma amostra de empresas de cada prédio do parque, além de compartilhar um mapa desenvolvido na mesma biblioteca que utilizávamos.

Com base nessas informações, coordenei com os AGES II uma alteração estrutural importante na entidade dos expositores, adicionando o atributo “prédio” para armazenar a identificação do edifício correspondente. Essa modificação permitiu vincular adequadamente as empresas aos seus respectivos prédios.

Durante a primeira semana, Alessandro obteve o mapa prometido pela equipe do Tecnopuc. A qualidade do material recebido superou nossas expectativas, pois já continha todos os prédios modelados detalhadamente. Após análise, desenvolvi uma estratégia técnica: separar cada prédio em arquivos individuais e remontá-los em uma nova cena. Essa abordagem preservaria a funcionalidade essencial de destacar visualmente os prédios conforme as buscas realizadas pelos usuários. A Figura 24 apresenta a versão final do mapa do Tecnopuc.

Figura 24 – Versão final do mapa do Tecnopuc



Fonte: Código-fonte do *frontend*

No *backend*, a única funcionalidade pendente era aquela que havia ficado como débito técnico da *sprint* anterior. Como estava sob responsabilidade de outro AGES III, optei por não intervir diretamente, confiando no processo estabelecido de acompanhamento.

Durante o desenvolvimento desta *sprint*, observei um fenômeno preocupante: um relaxamento generalizado da equipe, incluindo os próprios AGES IV. O sucesso alcançado até então parecia ter criado uma falsa sensação de conclusão, quando na realidade diversos ajustes importantes ainda demandavam atenção. Essa situação

resultou em acúmulo de tarefas para os momentos finais do projeto.

Na última semana, durante o *code freeze*, identifiquei funcionalidades críticas do *frontend* que permaneciam não implementadas, incluindo a integração do *endpoint* de popular o banco com a página de administração. Em discussão com o AGES III Eduardo Felber Eichner, estabelecemos uma divisão: ele finalizaria a integração da tela de *QR Code*, trabalho já iniciado na *sprint* anterior, enquanto eu assumiria a integração do *endpoint* de importação de dados.

A implementação transcorreu sem grandes obstáculos, pois a complexidade técnica era inferior aos desafios que havia enfrentado anteriormente. Desenvolvi uma tela intermediária, realizei a integração necessária e aguardei Eduardo concluir sua parte. Posteriormente, colaboramos na implementação do último elemento da interface administrativa: o botão de exportação de dados dos usuários e suas pesquisas no totem.

No dia da apresentação, dediquei-me a adicionar refinamentos que enriquecessem a experiência do usuário. Percebi que não havíamos implementado um método adequado para acessar a área administrativa. Solucionei essa lacuna criando um botão discreto que, ao ser acionado, exibia uma *modal* solicitando senha de acesso ao painel administrativo.

Durante a validação final com os AGES IV, fui informado sobre um serviço AWS implementado para geração dinâmica de *QR Codes* durante a apresentação. Nos testes, identifiquei uma incompatibilidade: o *frontend* esperava *QR Codes* com texto em português, enquanto o serviço gerava conteúdo em inglês. Rapidamente criei uma *branch* específica para a apresentação e implementei a correção necessária, garantindo o funcionamento perfeito do sistema.

A apresentação final superou todas as expectativas. Além do *stakeholder* Igor Daudt, contamos com a presença ilustre de Jorge Audy, um dos principais idealizadores do Tecnopuc. A demonstração não apenas impressionou nosso cliente original, mas despertou genuíno interesse do convidado especial. Jorge Audy compartilhou que o Tecnopuc enfrenta desafios significativos com a localização de auditórios, pois visitantes frequentemente se perdem entre os diversos prédios ao procurar salas de palestras.

Esta *sprint* final consolidou o sucesso do projeto, transformando nossa solução em

uma ferramenta com potencial real de aplicação. A validação por figuras importantes do ecossistema tecnológico confirmou a relevância e qualidade do trabalho desenvolvido. Apesar dos desafios enfrentados com o relaxamento da equipe e as correções de última hora, a capacidade de adaptação e resolução rápida de problemas garantiu uma entrega impecável, demonstrando a maturidade técnica alcançada ao longo do projeto.

4.4 Conclusão

Minha experiência como AGES III no projeto Totem Interativo representou um marco significativo em minha trajetória acadêmica. Pela primeira vez, assumi um papel de liderança técnica, sendo responsável não apenas pelo desenvolvimento, mas também pela orientação dos AGES I e II. Essa posição trouxe desafios únicos que contribuíram para meu amadurecimento profissional.

O domínio da biblioteca Three.js mostrou-se fundamental para o sucesso do projeto. A capacidade de criar mapas 3D interativos, desde versões simples até a complexa representação do Tecnopuc, permitiu que eu assumisse tarefas críticas e garantisse entregas de qualidade, superando as limitações de *hardware* dos totens.

Um dos maiores aprendizados foi sobre a dinâmica de equipe. A parceria com Eduardo Felber Eichner foi crucial para manter o progresso do projeto, mas reconheço que deveria ter sido mais assertivo ao comunicar preocupações sobre a distribuição desigual de responsabilidades entre os AGES III. Essa experiência destacou a importância da comunicação direta em projetos colaborativos.

Os contratempos técnicos, como problemas com o GitLab e correções de última hora, testaram minha capacidade de resolução rápida de problemas. Manter a calma e encontrar soluções sob pressão foi um aprendizado valioso para minha carreira.

O sucesso da apresentação final, com a validação do *stakeholder* e o interesse de Jorge Audy do Tecnopuc, confirmou a qualidade do trabalho desenvolvido. Ver nossa solução reconhecida como ferramenta com potencial real de aplicação foi extremamente gratificante.

Quanto à relação entre as disciplinas cursadas e a AGES, acredito que a disciplina de Projeto e Arquitetura de Software foi a que mais contribuiu para a minha atuação no projeto. Foi a partir dela que consegui compreender melhor as necessidades ar-

quiteturais do Totem Interativo, como a separação em camadas, a definição da API como fronteira entre a interface e os dados e as decisões necessárias para que a aplicação operasse localmente, sem depender de conexão externa. Esse embasamento me permitiu tomar decisões de arquitetura com mais segurança e orientar os demais integrantes da equipe. Por outro lado, uma crítica que faço é que, embora não tenha impactado de forma significativa, já que o projeto dispensava o uso da AWS, a disciplina de Construção de Software deixou a desejar no preparo para esse tipo de desafio de infraestrutura em nuvem, que poderia ter sido necessário em um cenário diferente.

Esta passagem pela AGES III consolidou conhecimentos técnicos em arquiteturas modernas e habilidades essenciais de liderança e trabalho em equipe. A experiência de revisar código, orientar implementações e tomar decisões arquiteturais preparou-me adequadamente para os desafios do mercado de trabalho, marcando minha evolução de estudante para engenheiro de software.

5 AGES IV — ESTUDIFY

Esta seção busca apresentar a passagem do aluno como AGES IV pelo Projeto Estudify. Aqui estão descritos os artefatos entregues, a atuação ao longo das sprints e os pontos de melhoria identificados.

5.1 Introdução

O projeto foi concebido a partir de uma "dor" identificada pelas *stakeholders* Eliza Pasquali Lunardi e Letícia Fossá Rosa, fundadoras do Estudify e vencedoras do Startup Garagem. Embora já possuíssem uma plataforma web funcional de preparação para o ENEM e vestibulares com usuários pagantes, as clientes enfrentavam limitações na experiência oferecida aos estudantes e dificuldades recorrentes na gestão de conteúdo, identificando a necessidade de evoluir o produto para uma aplicação *mobile* nativa, mais engajadora e alinhada ao comportamento do público jovem.

O projeto tem como objetivo tornar o estudo para o ENEM e vestibulares mais leve e eficiente, por meio de uma experiência gamificada inspirada no Duolingo. Foi proposto um aplicativo *mobile* em que o estudante aprende resolvendo questões organizadas por disciplina e tópico, com *feedback* instantâneo via gabarito comentado, complementado por simulados completos no formato ENEM, cronograma personalizado de estudos e um sistema de gamificação com moedas, *streaks* diários e acompanhamento de progresso. O projeto também contempla um painel administrativo web, destinado às clientes, para importação em lote de questões e acompanhamento dos usuários.

A execução do projeto ocorreu no primeiro semestre de 2026, entre os dias 02 de março e 29 de junho de 2026, compreendendo diversas etapas realizadas pelos estudantes do bacharelado em Engenharia de Software no âmbito da AGES, sob a orientação do professor Marcelo H. Yamaguti. A foto do time responsável pelo projeto pode ser vista na Figura 25.

Figura 25 – Time do projeto Estudify



Fonte: Wiki do projeto

5.2 Desenvolvimento do Projeto

Esta seção apresenta informações relativas ao desenvolvimento do projeto: localização do código-fonte e documentação, arquitetura e tecnologias utilizadas, e protótipos de tela desenvolvidos. Toda a documentação do projeto pode ser encontrada na wiki do mesmo: `https://tools.ages.pucrs.br/estudify/estudify-wiki/-/wikis/home`.

5.2.1 Repositório do Código Fonte do Projeto

Todo o código do projeto encontra-se distribuído em três repositórios:

- código do *frontend* web: `https://tools.ages.pucrs.br/estudify/frontend-web`;
- código do *frontend mobile*: `https://tools.ages.pucrs.br/estudify/frontend-mobile`;
- código do *backend*: `https://tools.ages.pucrs.br/estudify/backend`.

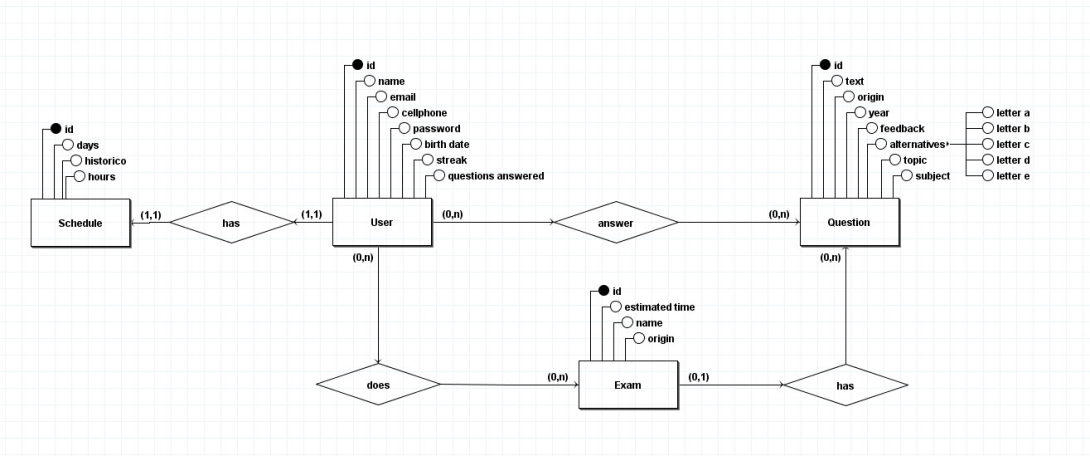
5.2.2 Banco de Dados Utilizado

O banco de dados escolhido foi o PostgreSQL, um banco de dados transacional que possui todas as propriedades Atomicity, Consistency, Isolation, Durability (ACID). A escolha se deu por sua robustez e confiabilidade, além de se tratar de um sistema de gerenciamento de banco de dados *open source* amplamente utilizado no mercado, o que facilita a sua adoção, manutenção e escalabilidade.

A modelagem foi desenvolvida de maneira relacional, o que facilitou o tratamento dos relacionamentos entre as entidades da aplicação. As principais entidades do banco são: *User*, que armazena as informações dos usuários, sejam eles administradores ou alunos; *Subject* e *Path*, que representam, respectivamente, as disciplinas e as trilhas de aprendizado que organizam seus conteúdos; *Question* e *Alternative*, que armazenam as questões e suas alternativas; *Exam*, que representa os simulados; *Attempt*, que registra as tentativas de realização dos simulados; *Answer*, que guarda as respostas dos usuários, permitindo o acompanhamento de desempenho; *StudyDay* e *StudyLog*, que definem e registram o cronograma de estudos de cada usuário; e *Subscription*, que gerencia as assinaturas e o controle de acesso aos recursos da plataforma.

A Figura 26 apresenta o modelo conceitual, que evidencia as entidades centrais do sistema e os relacionamentos entre elas em um nível mais abstrato.

Figura 26 – Modelo conceitual Estudify

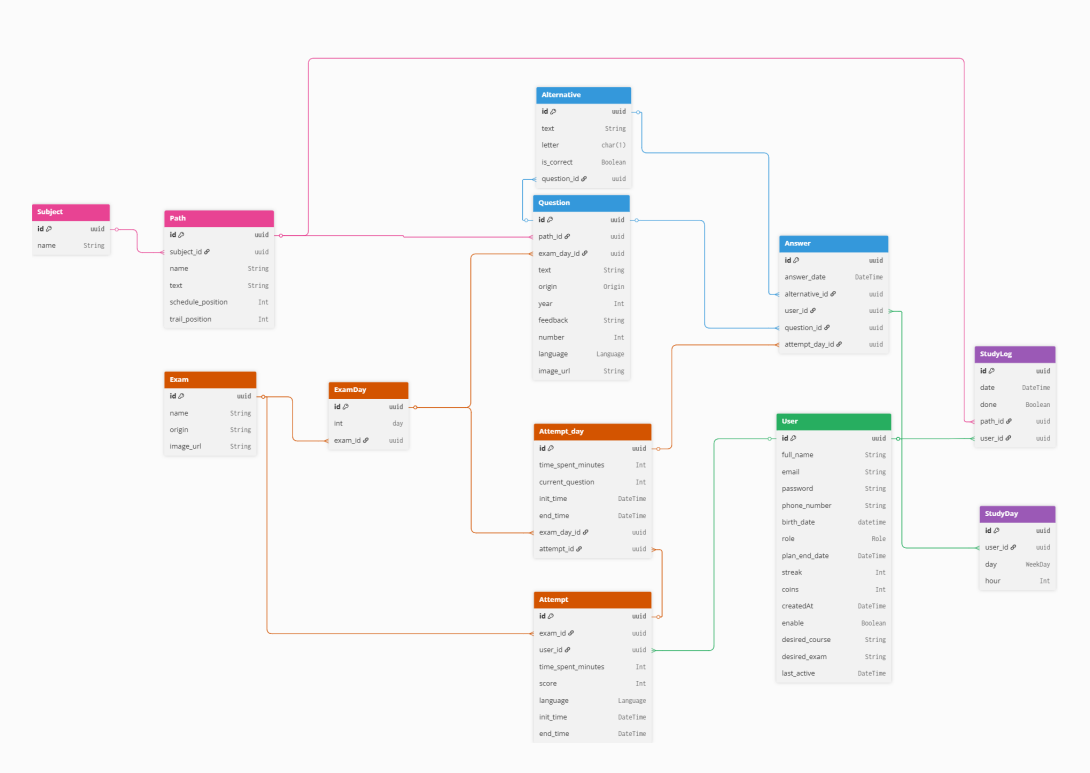


Fonte: Wiki do projeto

Já a Figura 27 apresenta o modelo lógico relacional, detalhando as tabelas, seus atributos e respectivos tipos, bem como as chaves primárias e estrangeiras que mate-

rializam os relacionamentos no banco de dados.

Figura 27 – Modelo lógico relacional Estudify



Fonte: Wiki do projeto

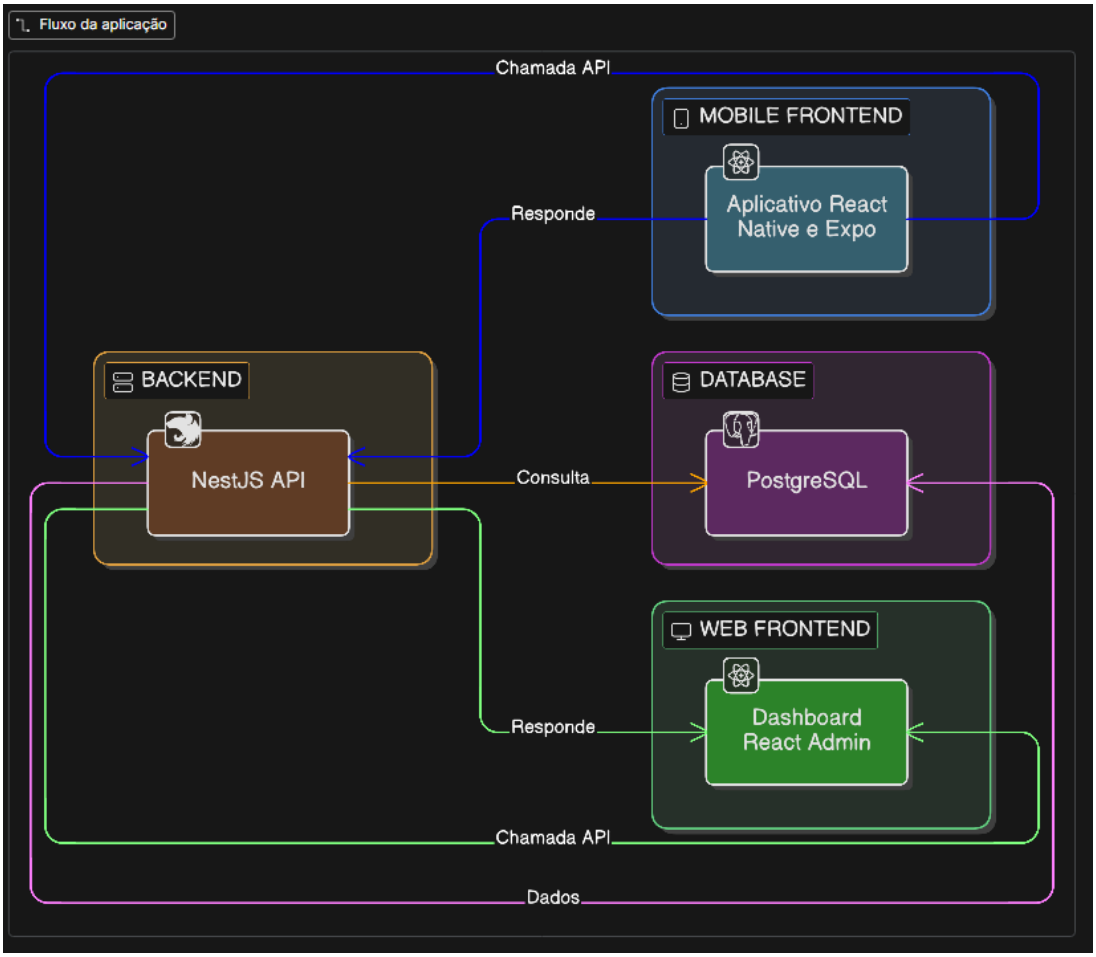
5.2.3 Arquitetura Utilizada

A arquitetura do projeto segue o padrão de Monólito Modular no *back-end*, em que as funcionalidades são segregadas por domínio. Cada módulo é composto por *Controllers*, responsáveis pelas rotas e requisições HTTP; *Services*, que concentram as regras de negócio; *Repositories*, camada isolada que utiliza o Prisma para a comunicação com o banco de dados; e Data Transfer Object (DTO) (*Data Transfer Objects*), que definem os contratos de entrada com validações integradas.

A aplicação *mobile* e o painel administrativo web adotam uma estrutura de pastas análoga entre si (*components*, *hooks*, *services* e *contexts*), mantendo a consistência técnica entre as frentes e permitindo que qualquer integrante do time atue em ambas.

O fluxo de comunicação entre as camadas está ilustrado no diagrama da Figura 28, evidenciando a separação entre os clientes (*mobile* e web), a API e o banco de dados.

Figura 28 – Diagrama de fluxo da aplicação



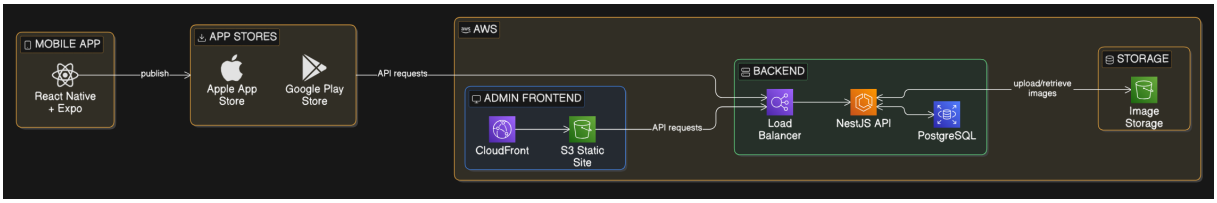
Fonte: Wiki do projeto

No que se refere à infraestrutura, a aplicação é hospedada na nuvem da AWS, que fornece serviços gerenciados voltados à disponibilidade, à segurança e à escalabilidade. O aplicativo *mobile*, desenvolvido em React Native com Expo, é compilado e publicado diretamente nas lojas oficiais (*App Store* e *Google Play Store*) e consome a API por meio de requisições Hypertext Transfer Protocol Secure (HTTPS). O painel administrativo web, por sua vez, é tratado como um site estático: os arquivos gerados no *build* do React são hospedados no *Amazon S3* e distribuídos pelo *CloudFront*, que atua como Content Delivery Network (CDN) e provê o certificado Secure Sockets Layer (SSL).

As requisições à API passam por um *Load Balancer*, responsável por distribuir o tráfego entre as instâncias da API em NestJS, mantendo o sistema operante mesmo em picos de acesso. A camada de persistência é composta pelo PostgreSQL, utilizado para os dados estruturados, e pelo *Amazon S3*, empregado no armazenamento de

arquivos mais pesados, como imagens. Esse isolamento entre o banco de dados e o armazenamento de arquivos permite que cada camada escale de forma independente. A Figura 29 apresenta o diagrama de *deploy*, evidenciando como esses componentes se comunicam.

Figura 29 – Diagrama de *deploy* da aplicação



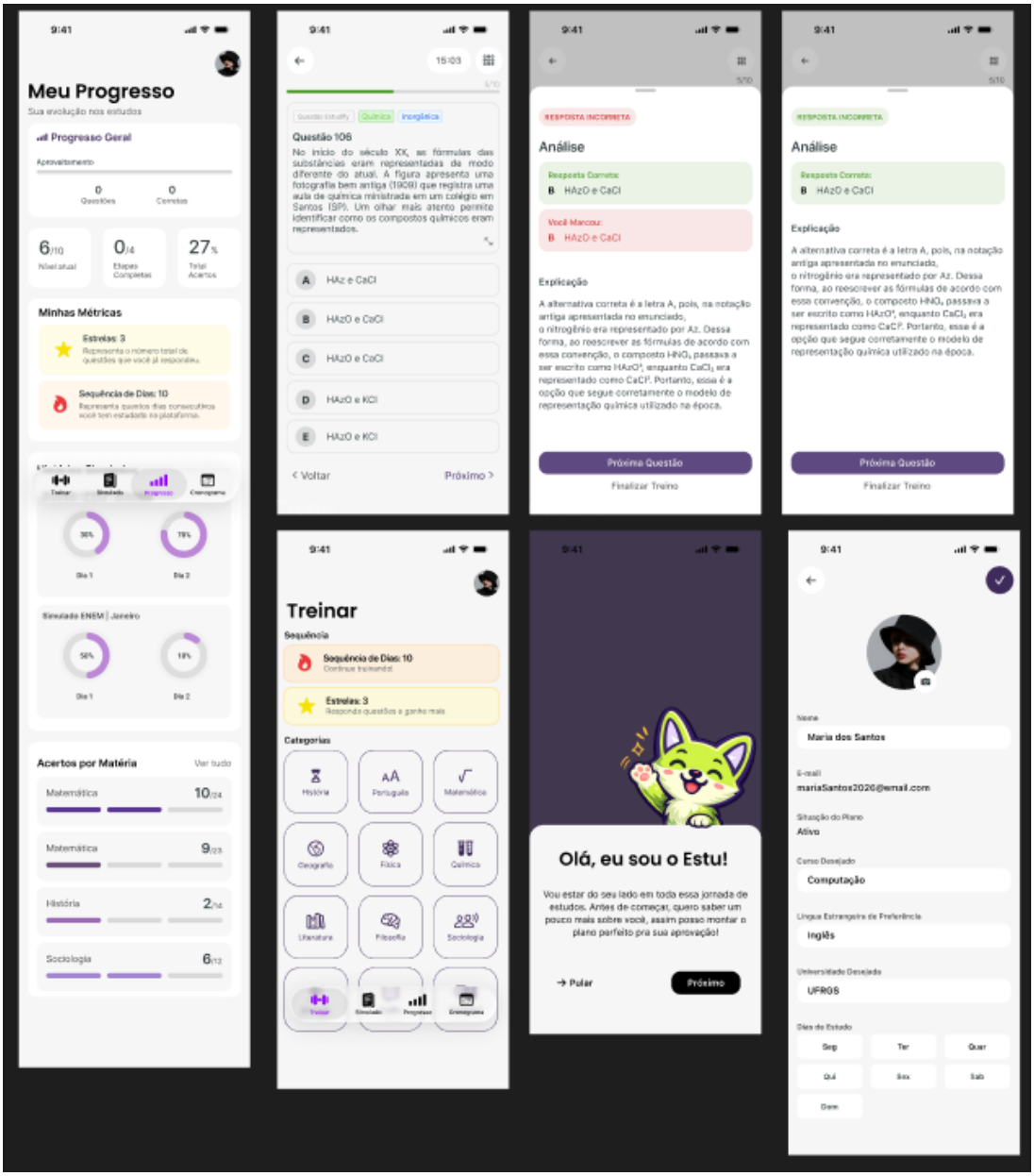
Fonte: Wiki do projeto

5.2.4 Protótipos das Telas Desenvolvidas

Os protótipos deste projeto estão disponíveis na wiki do projeto: `https://tools.ages.pucrs.br/estudify/estudify-wiki/-/wikis/design`.

A Figura 30 apresenta uma amostra das telas do aplicativo, contemplando os principais fluxos do usuário: o acompanhamento do progresso com os elementos de gamificação, a tela de treino com a seleção de disciplinas e tópicos, a resolução de questões e o respectivo *feedback* com a explicação da resposta, além do *onboarding* de primeiro acesso e da tela de perfil.

Figura 30 – Protótipos das telas do aplicativo *mobile*



Fonte: Wiki do projeto

Além do aplicativo *mobile*, o projeto conta com um painel administrativo web, destinado à gestão de conteúdo da plataforma. As Figuras 31, 32 e 33 apresentam as principais telas desse painel. A Figura 31 exibe a tela de gestão de questões, com busca, filtro por categoria e a listagem de todas as questões cadastradas. A Figura 32 apresenta a tela de gestão de simulados, em que é possível organizar os simulados e definir a ordem das questões que os compõem. Por fim, a Figura 33 mostra a tela de revisão das questões importadas em lote via CSV, sinalizando o *status* de cada uma (sucesso, falta de imagem ou erro) para que os ajustes necessários sejam feitos antes

da confirmação.

Figura 31 – Painel administrativo: gestão de questões



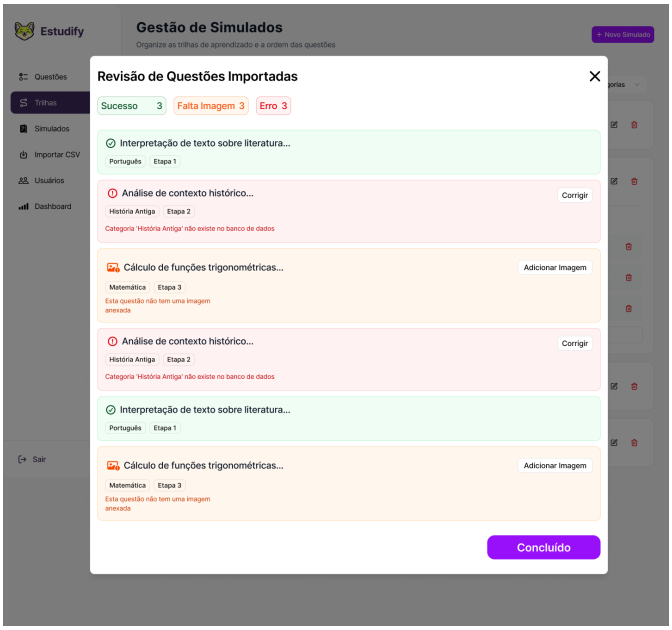
Fonte: Wiki do projeto

Figura 32 – Painel administrativo: gestão de simulados



Fonte: Wiki do projeto

Figura 33 – Painel administrativo: revisão de questões importadas via CSV



Fonte: Wiki do projeto

5.2.5 Tecnologias Utilizadas

Nesta seção serão apresentadas as principais tecnologias empregadas no desenvolvimento do projeto Estudify, destacando suas contribuições para o sucesso da implementação. As ferramentas estão organizadas conforme a camada da aplicação em que foram utilizadas.

- **TypeScript** (Microsoft, 2012): *Superset* do JavaScript que adiciona tipagem estática opcional, reduzindo *bugs* em tempo de desenvolvimento e facilitando a manutenção do código. Foi adotado como linguagem base em todas as camadas da aplicação (*back-end*, *mobile* e web), permitindo o compartilhamento de tipos e interfaces entre as frentes do projeto.

- **Node.js** (Node.js Foundation, 2009): Ambiente de execução JavaScript *server-side* com natureza assíncrona e eficiência no gerenciamento de I/O, ideal para o processamento de múltiplas requisições simultâneas. Serve como base para a execução do NestJS.

- **NestJS** (NestJS, 2017): *Framework* progressivo para Node.js que permite a construção de aplicações escaláveis com arquitetura modular e suporte nativo

a TypeScript. Foi utilizado na construção da API do projeto, oferecendo recursos como injeção de dependência, *decorators* e validação integrada de DTO.

- **Prisma** (Prisma, 2021): ORM (*Object-Relational Mapper*) que atua como ponte entre a aplicação e o banco de dados relacional. Gera tipos automaticamente a partir do *schema* do banco, garantindo *type-safety* nas consultas, além de simplificar a criação e execução de *migrations*.

- **PostgreSQL** (PostgreSQL Global Development Group, 1996): Sistema de gerenciamento de banco de dados relacional *open source* que implementa as propriedades ACID, garantindo a integridade e a consistência dos dados armazenados. Escolhido por sua robustez, confiabilidade e maturidade no mercado.

- **React Native** (Meta, 2015): *Framework* para desenvolvimento de aplicações *mobile* nativas a partir de uma única base de código em TypeScript, permitindo a entrega simultânea para iOS e Android. Foi utilizado na construção do aplicativo principal do Estudify.

- **Expo** (Expo, 2015): Plataforma e ecossistema construído sobre o React Native que agiliza o desenvolvimento *mobile*, fornecendo ferramentas integradas de *build*, execução e distribuição, além do Expo Router para roteamento baseado em arquivos.

- **React** (Meta, 2013): Biblioteca JavaScript de código aberto para construção de interfaces de usuário reativas e componentes reutilizáveis. Foi utilizada no desenvolvimento do painel administrativo web, destinado à gestão de conteúdo e acompanhamento dos usuários.

5.3 Atividades Desempenhadas Pelo Aluno no Projeto

Nesta seção será abordada a minha contribuição para o projeto ao longo das *sprints*.

5.3.1 Sprint 0

O projeto teve início com uma reunião com as *stakeholders* Eliza Pasquali Lunardi e Letícia Fossá Rosa, onde tivemos a oportunidade de entender com mais profundidade

do que se tratava o Estudify. Nesse encontro, tiramos dúvidas de escopo e prioridades, e alguns ajustes importantes emergiram em relação ao material de abertura do projeto: a revisão de redação, que constava como item do escopo, acabou ficando de fora, enquanto a página web de administração, inicialmente listada como fora do escopo, foi incorporada. Saímos da reunião com bastante material para trabalhar ao longo da *sprint*.

Logo em seguida, nos reunimos entre os AGES IV para alinhar como conduziríamos o projeto. Já no primeiro dia, debatemos alguns princípios que nortearam toda a nossa atuação. Definimos que as *tasks* seriam escritas de forma bem mais detalhada do que o padrão — uma decisão baseada no nosso histórico na AGES, que mostrava que tarefas mal definidas designadas a AGES I ou II tendiam a gerar retrabalho significativo. Também definimos que trabalharíamos organizados em *squads*, compostas por pelo menos um integrante de cada senioridade, e que cada AGES IV ficaria responsável por acompanhar uma delas, com a missão de garantir que o time avançasse nas entregas. Essa divisão nos deu um controle muito mais assertivo sobre o andamento do projeto.

Para fundamentar a escolha do *stack*, elaboramos uma pesquisa interna mapeando as tecnologias mais familiares ao time. Deixamos claro desde o início que a decisão final caberia aos AGES III, por serem eles os responsáveis pela arquitetura. Após a discussão técnica, definiu-se React Native para o *mobile* e NestJS para o *backend*; em conversa com os AGES II, que responderiam pelo banco, fechamos em PostgreSQL.

Quando conversamos com os AGES III sobre a distribuição de responsabilidades na *sprint*, dois deles, Eduardo Camana e Isadora Guerra, manifestaram interesse em se dedicar inteiramente à prototipação. Ambos vinham do Apple Academy, e como eu já havia tido experiências muito positivas com a prototipação de quem participa deste programa, decidimos aceitar. Ainda assim, abrimos o convite para qualquer outro integrante do time que quisesse aprender mais sobre essa frente se juntar a eles. Os três AGES III restantes assumiram a montagem da infraestrutura. Esse tipo de paralelismo só foi viável porque tínhamos cinco AGES III, um número acima da média para projetos da AGES, e uma equipe muito engajada no geral.

Em paralelo, estruturamos o escopo do aplicativo em nove épicos, cada um agru-

pando um conjunto coeso de US. Fiquei responsável por escrever as US de dois deles: o Épico 3, que concentrava o *core* do produto, toda a jornada de resolução de questões, e o Épico 6, referente à gamificação do aplicativo como um todo. Reforçamos entre nós a disciplina de escrever *tasks* o mais detalhadas possível, com critérios de aceite claros e contratos bem definidos entre *front* e *back*. Quando terminamos de redigir as histórias, nos reunimos como AGES IV e revisamos uma a uma, garantindo que cada ponto mapeado no escopo tivesse ao menos uma *task* correspondente no ClickUp.

Um ponto que não previmos foi o ritmo de entrega dos AGES II no modelo de banco. Nossa expectativa era de que um modelo minimamente concreto só apareceria por volta do meio da *Sprint* 1, e estruturamos a escrita das US partindo dessa premissa. Os AGES II, no entanto, conseguiram entregar um modelo bastante completo já no fim da *Sprint* 0, o que, apesar de ser um ótimo resultado de engajamento da equipe, fez com que algumas das histórias que escrevemos ficassem ligeiramente desalinhadas em relação ao modelo que chegou. Não foi um erro muito significativo, mas gerou um ruído extra no início da *Sprint* 1 que poderia ter sido evitado. Ficou como aprendizado, para mim como AGES IV, a necessidade de sincronizar mais de perto a escrita das US com o avanço da modelagem de dados, inclusive nos cenários em que o time técnico entrega acima do esperado, o descompasso pode vir pelos dois lados.

Na apresentação de fechamento da *sprint*, as *stakeholders* ficaram bastante satisfeitas com os protótipos no geral; o resultado foi muito além do que haviam imaginado. Solicitaram apenas alguns ajustes pontuais na UI, como trocar a fonte por uma com traços mais arredondados e deixar as telas mais coloridas. Ficaram também de nos enviar alguns ícones que gostariam de ver aplicados no aplicativo, bem como uma lista de questões já existentes no banco de dados delas, que serviria de base para o conteúdo inicial.

Ao final da *sprint*, senti que o time em geral fez um excelente trabalho. O único ponto de atenção que levantei junto aos outros AGES IV foi que os AGES I acabaram tendo pouca participação nessa *sprint*: como normalmente são eles que contribuem com os *mockups* nessa fase, e essa frente ficou a cargo do Eduardo e da Isadora, sobrou menos espaço de atuação para eles. Não foi um problema de engajamento,

mas algo a observar nas próximas *sprints* para garantir que essa senioridade tenha frentes claras em que possa contribuir.

5.3.2 Sprint 1

Após a apresentação com as *stakeholders*, reunimos o time para dividir as *squads*. Para esta *sprint*, planejamos doze tarefas de desenvolvimento, sendo seis referentes ao Épico 3, de questões, e seis distribuídas entre os demais épicos. Como o número de entregas era menor que o de integrantes, algumas delas acabaram sendo feitas em dupla, normalmente com um AGES I e um AGES II trabalhando juntos. O meu time ficou responsável pelas tarefas de autenticação e cadastro do usuário. A distribuição das histórias dentro de cada *squad* ficou a cargo do AGES IV responsável por ela.

Um detalhe importante é que, antes mesmo do início da *sprint*, um dos AGES III já havia adiantado a parte de autenticação no *backend*, cobrindo tanto o login quanto o cadastro do usuário. Isso fez com que essa frente já chegasse praticamente concluída na minha *squad*. Entreguei para o AGES II do time a tarefa de *frontend* de cadastro, e para o AGES I o carrossel de primeiro acesso ao aplicativo. Como meu AGES III já tinha adiantado parte do *backend*, optei por passar a ele uma atividade menos complexa, o que permitiria que ele apoiasse os AGES menores, revisasse os PR e começasse a desenhar o diagrama da AWS, uma entrega que precisaria sair o quanto antes.

Logo após a primeira aula da *sprint*, percebemos que o engajamento do time estava bastante alto. Já havia PR abertos para a *develop* nos primeiros dias. Com a capacidade sobrando, começamos a puxar histórias da *Sprint* 3, já que boa parte das entregas da *Sprint* 2 dependia diretamente de cadastro, login e questões ainda em andamento na *Sprint* 1. Antecipar histórias da *Sprint* 3 era mais seguro, pois elas tinham menos acoplamento com o que estava sendo construído naquele momento.

No meu time, no entanto, notei que as coisas andavam um pouco mais devagar, especialmente na tarefa do AGES II, que estava enfrentando dificuldades com o próprio Git. Conversei individualmente com cada integrante da *squad* para entender se havia algo em que eu pudesse ajudar. Todos responderam que estava tudo certo e que as atividades estavam encaminhadas.

Na aula seguinte, conversei com a AGES I da *squad* e ela me disse que a ta-

refa dela estava pronta, mas que tinha dúvida se deveria subir mesmo faltando uma rota. No fim, a rota dependia de uma tela de outra *squad* que acabou atrasando, o que bloqueou parcialmente a entrega dela, ainda que a parte que cabia à AGES I já estivesse pronta. Já o AGES II tinha um PR aberto com comentários dos AGES III pedindo ajustes, todos pontuais e resolvidos em aula. A única pendência real que ficou foi a história do AGES III do time, que acabou atrasando. Como era uma entrega importante, o componente de navegação do aplicativo, que destravaria várias atividades futuras, pedi que outro AGES III pegasse a tarefa antes do *code freeze*, que havíamos estipulado para uma semana antes da apresentação. O Gabriel Saldanha, de outra *squad*, puxou a história para si e conseguiu finalizá-la faltando três dias para a apresentação, já dentro do *code freeze*.

Na aula anterior à apresentação, tentamos rodar um ensaio interno do que havia sido desenvolvido ao longo da *sprint*, mas não tivemos sucesso. O repositório do *mobile* estava com erro ao tentar emular o app no Expo. Mais uma vez, o Gabriel Saldanha se mostrou prestativo e ficou responsável por ajustar o problema.

Depois dessa aula, me reuni com os outros AGES IV para montar a apresentação e fazer um último teste no fluxo completo. Foi nesse momento que notamos uma quantidade considerável de *bugs* que haviam passado para a *develop*, alguns bastante escancarados. Em caráter de emergência, acionamos todos os AGES III para verificar o que conseguiriam corrigir até a apresentação. No fim, a maior parte dos *bugs* foi resolvida, mas alguns persistiram. Anotamos como ponto de ação a criação de histórias específicas para resolução desses *bugs* na *sprint* seguinte.

Na apresentação, mostramos às *stakeholders* tudo o que havia sido desenvolvido e reforçamos com elas o envio dos materiais que ainda estavam pendentes desde a *Sprint* 0, os ícones do aplicativo e a base de questões. Elas ficaram bastante satisfeitas com o que viram e se comprometeram a enviar o material em seguida.

Apesar dos contratempos, conseguimos entregar mais atividades nesta *sprint* do que havíamos planejado, o que reforçou a leitura de que a maior parte do time estava bem engajada com o projeto.

Após a apresentação, fizemos a retrospectiva da *sprint*. Muitos pontos positivos foram levantados em relação à organização e à comunicação entre os times, mas também apareceram pontos de atenção. Como já havia sido comentado na *Sprint* 0,

algumas histórias não estavam totalmente alinhadas com o modelo de banco, o que acabou gerando dúvidas durante o desenvolvimento, e algumas regras de negócio não ficaram explícitas o suficiente nas US. Criamos algumas pendências internas entre os AGES IV para serem resolvidas até a próxima aula. Ficou combinado que cada AGES IV revisaria as histórias da *Sprint* 2 do seu time e solicitaria uma segunda revisão de outro AGES IV, para garantir que as US chegassem ao time com o nível de clareza esperado.

Como AGES IV, tive dificuldade em acompanhar de perto o AGES III da minha *squad* nos momentos finais da *sprint*. Nos dias que antecederam o *code freeze*, tentei contato algumas vezes para entender como estava o andamento da história e oferecer apoio, mas tive pouco retorno, o que atrasou a decisão de redistribuir a entrega e acabou se tornando um risco concreto para a *sprint*. Outro ponto que só ficou claro ao longo dessas semanas foi a dificuldade de planejar um projeto inteiro no período curto que temos, algo que impacta tanto o detalhamento das histórias quanto a previsão de dependências entre as frentes.

5.3.3 Sprint 2

Antes mesmo do início da *sprint*, eu e os demais AGES IV do time revisamos todas as tarefas que já haviam sido criadas, de modo que ficassem alinhadas tanto com as regras de negócio quanto com o modelo de banco de dados, facilitando o desenvolvimento. Dividimos entre duas e três tarefas para cada um revisar, conforme a complexidade. Enquanto revisava as que ficaram sob minha responsabilidade, referentes à execução de simulados, percebi no código um detalhe do *frontend* que poderia impactar a *sprint*: alguns componentes que deveriam ter sido criados na *sprint* anterior, para serem reutilizados nesta, haviam sido implementados de forma *hard coded*, e não como componentes reaproveitáveis. Avisei os demais AGES IV do time e ajustei a tarefa desta *sprint* para que isso fosse corrigido, seguindo as boas práticas de engenharia de software.

Nesta *sprint*, definimos que as entregas seriam a outra *feature* central da aplicação, a parte de criação e execução de simulados, além do *onboarding* do usuário e do bloqueio de pagamentos (*paywall*). Para a execução de simulados, optamos por reutilizar parte do código já desenvolvido para as questões, já que, na prática, ambos

cumprem o mesmo propósito. A única diferença é que, nas questões de treino, o usuário recebe o *feedback* de forma imediata, enquanto no simulado isso ocorre apenas ao finalizá-lo.

Decidimos também alterar um pouco a metodologia de trabalho. Em vez de distribuir tarefas específicas para cada *squad*, passamos a distribuí-las de forma individual, deixando a *squad* apenas como um norte, indicando a quem a pessoa deveria recorrer primeiro caso precisasse de ajuda. Assim, no meu time, as tarefas ficaram divididas entre os épicos: pegamos atividades de execução de simulados, de *paywall* e algumas telas do *dashboard* de administração.

No início da *sprint*, tivemos um problema de comunicação no time. A forma como a tabela de simulados seria utilizada foi alterada, mas a mudança demorou bastante para ser *mergeada* no repositório, o que gerou retrabalho, pois as demais tarefas já estavam em desenvolvimento quando souberam dessa informação. Por conta disso, junto com os AGES III e o pessoal que estava desenvolvendo as tarefas de simulados, reformulamos alguns pontos da regra de negócio: removemos o estado de “pausado” e passamos a permitir apenas um simulado ativo por *id* de simulado. Ou seja, se um usuário possui um simulado ativo e deseja refazê-lo, o *backend* entende que ele está encerrando a tentativa em aberto e iniciando uma nova.

Ainda na primeira semana de desenvolvimento, junto com os AGES III responsáveis pela UI/User Experience (UX) do projeto, marcamos uma reunião com as *stakeholders* para apresentar o que havíamos imaginado para as telas que elas apontaram precisar de ajustes: a tela inicial de treino, em que o usuário escolhe a disciplina, e a tela de trilha, em que ele escolhe o tópico. Elas aprovaram as alterações, mas, como o escopo desta *sprint* já estava definido, optamos por não as incorporar agora, deixando-as para a *Sprint* 3.

Entre o meio e o fim da *sprint*, precisei me ausentar das aulas por conta de uma cirurgia. Ainda assim, procurei manter contato tanto com o meu time quanto com os demais AGES IV para acompanhar o andamento do projeto. Quando retornei, estávamos bastante atrasados para a entrega, já nos últimos dias de *sprint*. Em tese, o *code freeze* já deveria ter ocorrido, mas, se o respeitássemos, não teríamos nada a entregar. Por isso, decidimos que cada um enviaria o que conseguisse finalizar e os AGES III iriam corrigindo conforme fosse possível.

Na terça-feira, véspera da apresentação, me reuni com os demais AGES IV para preparar a apresentação, e foi quando notamos que quase todas as tarefas entregues estavam com *bugs*. Como ainda estava afastado do trabalho e estava com bastante tempo disponivel para me dedicar ao projeto naquele período, prontifiquei-me a corrigir o que fosse possível até a hora da apresentação. Durante todo o dia da apresentação para as *stakeholders*, fiz ajustes no código para que conseguíssemos entregar algo a elas. No fim, consegui corrigir boa parte dos *bugs* e, mais uma vez, as *stakeholders* ficaram bastante satisfeitas com a entrega.

Na retrospectiva, surgiram vários pontos sobre o *code freeze* não ter sido respeitado, sobre a demora na correção dos PR por parte dos AGES III e sobre a alteração no banco de dados, que demorou a ser *mergeada* na *develop* e não foi devidamente comunicada aos demais colegas. Também foi trazido que os AGES III estavam se sentindo sobrecarregados com a quantidade de tarefas e revisões que precisavam fazer. Com base nisso, junto com a equipe, montamos alguns pontos de ação para evitar que esses problemas voltassem a ocorrer. Estabelecemos que, sempre que algo fosse alterado no banco de dados de forma que pudesse influenciar o trabalho de outras pessoas, isso deveria ser notificado no Discord, no canal designado para o banco de dados. Também avisamos aos AGES III que iríamos rever como poderíamos reduzir essa sobrecarga deles nas próximas *sprints*. Por fim, reforçamos que todo o código de *frontend* deveria ser testado utilizando o Expo, e não apenas no navegador, pois assim, durante o desenvolvimento, conseguiríamos ter uma ideia melhor de como ele ficaria no celular.

5.3.4 Sprint 3

Levando em consideração o pedido dos AGES III, decidimos que o melhor a fazer nesta *sprint* seria deixá-los sem nenhuma tarefa definida, de modo que pudessem pegar a tarefa que quisessem, caso quisessem. Nesta *sprint*, voltamos a adotar o método de *squads*, com tarefas que se complementam.

Novamente, junto com os outros AGES IV, decidimos revisar cada história da *sprint*. Estava planejado entregar toda a parte do portal de administrador, em que seria possível adicionar simulados e também questões em lote; o cronograma, em que o usuário poderia escolher os dias e horários em que gostaria de estudar; a parte de *feedback*

gamificado, utilizando *streak* de dias e gratificando o usuário a cada questão respondida corretamente; e, por fim, foi adicionada também uma nova tarefa de simulados, para visualizar o histórico dos simulados já realizados pelo usuário.

Para evitar os problemas que tivemos nas duas *sprints* anteriores, em que alguns detalhes das tarefas não estavam totalmente alinhados, fizemos uma última revisão, agora com o time todo, para que pudessem pontuar o que poderíamos alterar. Isso nos trouxe alguns *feedbacks*, e nos prontificamos a fazer as mudanças, especialmente nas tarefas do meu time, que ficou responsável pelo épico de cronograma. Um dos nossos AGES III apontou que a tarefa de *backend* poderia ficar muito grande caso mantivéssemos tudo junto. Com esse retorno, decidi dividi-la em duas etapas: a primeira contemplaria apenas um CRUD básico para inserção e visualização do cronograma, enquanto a segunda concentraria toda a regra de negócio, conforme imaginávamos que o cronograma deveria funcionar.

No início da *sprint*, entrei em contato com o AGES I que havia ficado com a tarefa de *backend*, e ele me informou que estava com alguns problemas, mas que tentaria iniciá-la. Pouco depois, no entanto, avisou que precisaria se ausentar das aulas por um tempo. Isso trouxe certa preocupação quanto à entrega do cronograma, pois, sem o *backend*, ela não seria possível. No mesmo dia, em aula, conversei com o AGES III do meu time para que ele assumisse a tarefa e a desenvolvesse.

Durante esta *sprint*, eu também peguei uma tarefa para desenvolver, pois era uma entrega importante para as clientes e não daria tempo de concluí-la caso contrário. Fiquei responsável por refatorar a tela da trilha de tópicos, conforme havíamos combinado com as *stakeholders*. O desenvolvimento da tela em si não apresentou nenhum problema, exceto por um ponto de que não nos demos conta no início: para cada tópico existente, seria necessário criar um ícone que o diferenciasse, o que acabou levando bastante tempo.

Uma semana antes da apresentação, data do nosso *code freeze*, já tínhamos todas as tarefas aguardando a revisão dos AGES III, o que nos deu, como AGES IV, o indicativo que queríamos: o time havia produzido bem, e agora faltavam apenas a aprovação e correções pontuais no código. Esse processo, porém, acabou atrasando. Mesmo após passar a semana inteira cobrando, os PR só começaram a ser aprovados com mais frequência faltando três dias para a apresentação. No fim, dei suporte aos

AGES III na correção dos *bugs* nos últimos dias da *sprint*. Foi nesse período que notei um *bug* simples em um *endpoint* de administrador: o *backend* estava esperando receber o Universally Unique Identifier (UUID) de disciplinas e tópicos, o que não fazia sentido com a regra de negócio que havíamos montado. Como isso já havia sido *mergeado* na *develop*, criei uma tarefa de *bug* para que esse comportamento fosse corrigido na *sprint* seguinte.

A apresentação para as *stakeholders* ocorreu bem, porém, ao final, identificamos um *bug* na tela de progresso, onde fica a parte de gamificação: ela estava considerando apenas as questões que o usuário havia respondido, e não todas as questões do simulado, como era o esperado. Infelizmente, foi uma das *stakeholders* que percebeu. Anotamos como *bug* para ser resolvido na *sprint* seguinte.



REFERÊNCIAS

GOOGLE. **Flutter**. [*S. l.: s. n.*], 2017. Disponível em: https://flutter.dev/. Acesso em: 16 jun 2023.

GOOGLE. **Firebase**. [*S. l.: s. n.*], 2011. Disponível em: https://firebase.google.com/. Acesso em: 16 jun 2023.

GRIESEMER, Robert; THOMPSON, Ken; PIKE, Rob. **GoLang**. [*S. l.: s. n.*], 2009. Disponível em: https://go.dev/. Acesso em: 16 jun 2023.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL**. [*S. l.: s. n.*], 1996. Disponível em: https://www.postgresql.org/. Acesso em: 30 jun 2024.

TYPEORM. **TypeORM**. [*S. l.: s. n.*], 2016. Disponível em: https://typeorm.io/. Acesso em: 30 jun 2024.

AMAZON. **Amazon Web Services**. [*S. l.: s. n.*], 2006. Disponível em: https://aws.amazon.com/. Acesso em: 30 jun 2024.

MICROSOFT. **TypeScript**. [*S. l.: s. n.*], 2012. Disponível em: https://www.typescriptlang.org/. Acesso em: 19 jun 2025.

NESTJS. **NestJS**. [*S. l.: s. n.*], 2017. Disponível em: https://nestjs.com/. Acesso em: 19 jun 2025.

META. **React**. [*S. l.: s. n.*], 2013. Disponível em: https://react.dev/. Acesso em: 30 jun 2024.

SASS. **Sass: Syntactically Awesome Style Sheets**. [*S. l.: s. n.*], 2006. Disponível em: https://sass-lang.com/. Acesso em: 30 jun 2024.

THREE.JS. **Three.js**. [*S. l.: s. n.*], 2010. Disponível em: https://threejs.org/. Acesso em: 19 jun 2025.

VERCEL. **Next.js**. [*S. l.: s. n.*], 2016. Disponível em: https://nextjs.org/. Acesso em: 19 jun 2025.

NODE.JS FOUNDATION. **Node.js**. [*S. l.: s. n.*], 2009. Disponível em: https://nodejs.org/. Acesso em: 19 jun 2025.

OPENAPI INITIATIVE. **Swagger**. [*S. l.: s. n.*], 2011. Disponível em: https://swagger.io/. Acesso em: 19 jun 2025.

SQLITE. **SQLite**. [*S. l.: s. n.*], 2000. Disponível em: https://www.sqlite.org/. Acesso em: 19 jun 2025.

PRISMA. **Prisma ORM**. [*S. l.: s. n.*], 2021. Disponível em: https://www.prisma.io/. Acesso em: 07 jun 2026.

META. **React Native**. [*S. l.: s. n.*], 2015. Disponível em: https://reactnative.dev/. Acesso em: 07 jun 2026.

EXPO. **Expo**. [*S. l.: s. n.*], 2015. Disponível em: https://expo.dev/. Acesso em: 07 jun 2026.

AGES. **Wiki do projeto Focus**. [*S. l.: s. n.*], 2023. Disponível em: https://tools.ages.pucrs.br/projeto-focus/wiki/-/wikis/home. Acesso em: 16 jun 2023.

AGES. **Wiki do Decola**. [*S. l.: s. n.*], 2024. Disponível em: https://tools.ages.pucrs.br/plataforma-onboarding-para-novos-colaboradores/decola-wiki/-/wikis/home. Acesso em: 16 jun 2023.

AGES. **Wiki do Totem Interativo**. [*S. l.: s. n.*], 2025. Disponível em: https://tools.ages.pucrs.br/totem-interativo/wiki/-/wikis/home. Acesso em: 19 jun 2025.

AGES. **Wiki do projeto Estudify**. [*S. l.: s. n.*], 2026. Disponível em: https://tools.ages.pucrs.br/estudify/estudify-wiki/-/wikis/home. Acesso em: 07 jun 2026.

KHAN, Shaiq. **Exploring Asynchronous Programming In Dart & Flutter**. [*S. l.: s. n.*], 2023. Disponível em: https://medium.flutterdevs.com/exploring-asynchronous-programming-in-dart-flutter-25f341af32f. Acesso em: 19 jun 2023.

MICHAEL, Aiiyemi. **Mastering Asynchronous Programming in Flutter with Dart's async, await and future keywords**. [*S. l.: s. n.*], 2023. Disponível em: https://www.linkedin.com/pulse/mastering-asynchronous-programming-flutter-darts-async-michael. Acesso em: 19 jun 2023.

FIELD, Dylan. **Figma**. [*S. l.: s. n.*], 2016. Disponível em: https://www.figma.com/. Acesso em: 16 jun 2023.